

Environmental Features and Crash Frequency in Philadelphia

A street-segment study of Philadelphia crash frequency, road design, tree canopy, grade, and related public data. It documents the research question, source data, modeling findings, exploratory data-processing decisions, and the handoff from notebook analysis to the browser map in `app/`.

The notebook does not reproduce every app implementation detail. The [final web app](#) uses DuckDB-WASM to read exported GeoParquet files and run the map/story queries locally in the browser.

Executive Summary

This project analyzes automobile crash data in Philadelphia, PA to determine if there's a correlation between crashes and road grade (elevation) or tree canopy. Both vector data (e.g. PennDOT crashes, road geometry) and raster data (elevation and tree canopy coverage) were used. To prepare the data, a significant amount of effort was dedicated to determining the width and grade of city road segments. Post-processing, statistical analysis and regression were run to identify any correlation between crashes and grading or tree cover across road segments.

For road widths, curbs and cartway data were joined to centerline data. Transects were drawn perpendicular to centerlines and measured until hitting curb geometry, handling edge cases due to intersections or divided roads with islands. Road grading was calculated by sampling points along a centerline, obtaining corresponding values in elevation raster data and then taking the median. The resulting widths seem reasonable for much of Philadelphia's roads, though outliers with impossible grades were identified. Most suspicious segments were *near* large elevation changes, a data processing error that needs to be fixed.

Using negative binomial models, three regressions were run: non-natural factors vs crashes per centerline; then adding natural factors vs crashes per centerline; then natural factors with interactions vs crashes per centerline. Each successive model proved more robust, with the third demonstrating tree canopy coverage's apparent protective relationship was strongest on narrower streets and weakened as cartway width increased. Grade was associated with fewer crashes at lower-speed contexts, but the grade-speed interaction was positive, suggesting steepness becomes less protective, and potentially hazardous, as speeds rise.

It should be noted that data here has not been vigorously tested for accuracy. This project's assertions are preliminary.

Note on Process and Rejected Routes

This notebook keeps the successful data-processing path in executable code and summarizes rejected routes in short notes. Commented-out code is only retained where it documents an attempted route that was not carried into the final workflow.

Environment and Data Expectations

The notebook expects the `roads` Python environment and the local project data folders:

- `raw-data/` for downloaded source datasets.
- `transformed-data/` for projected and intermediate geospatial files.
- `philly_segments.parquet` and `philly_block_groups.parquet` for the app-ready exports.

Core Python packages used here include `pandas`, `geopandas`, `numpy`, `rasterio`, `matplotlib`, `seaborn`, `pygris`, `census`, and `requests`. Census API access should be provided with `CENSUS_API_KEY`; do not hardcode a key in the notebook.

```
In [1]: import pandas as pd
import geopandas as gpd
import numpy as np
import rasterio

from rasterio.plot import show
import matplotlib.pyplot as plt
import seaborn as sns
import shapely
from shapely import make_valid
from shapely.geometry import LineString, Point
from shapely.ops import unary_union

import pygris
import census

import requests
import os

import statsmodels.api as sm
import statsmodels.formula.api as smf
from scipy.stats import chi2

# transforms df to 2272 and writes it to specified filepath
# if filepath exists, read it and return
def to_2272_file(df, filepath):
    df_2272 = None;

    if not os.path.exists(filepath):
        print("write file")
        df_2272 = df.to_crs(2272)
```

```

        df_2272.to_file(filepath, driver="geoJSON")
    else:
        print("read file")
        df_2272 = gpd.read_file(filepath)

    return df_2272

crash_data = gpd.read_file("raw-data/PennDOT Crash Data/collision_crash_2020

```

Data Acquisition and Setup

2024 PennDOT Crash Data for Philadelphia

The crash data comes from the Philadelphia subset of PennDOT annual crash records published through OpenDataPhilly. This section checks severity fields and basic completeness before joining crashes to street segments.

Data Quality

What does `max_severity_level` mean?

0 - No injury

1 - Fatal injury 2 - Suspected serious injury - obvious because the person cannot move or operate normally 3 - Suspected minor injury - evident to others, but not as serious as above: bruises, bleeding, etc. 4 - Possible injury - not evident but reported; could include being knocked out or internal injuries

8 - Injury, unknown severity 9 - Unknown

We're going to need to finesse this a bit. We could shuffle "No Injury" to 5 and have a severity scale. That kind of works, though "Possible injury" gives me a bit of pause because it could seemingly be as severe as 3, perhaps 2.

We could try to scale, but I'm leaning more toward using these as categorical rather than ordinal.

```

In [2]: print(crash_data.columns)
        print(crash_data.crs) # 4326

        print(crash_data["max_severity_level"].describe())
        print(crash_data["max_severity_level"].value_counts(dropna=False))

        sns.countplot(x="max_severity_level", data=crash_data)
        plt.show()

```

```
Index(['objectid', 'crn', 'arrival_tm', 'automobile_count',
      'belted_death_count', 'belted_susp_serious_inj_count', 'bicycle_count',
      'bicycle_death_count', 'bicycle_susp_serious_inj_count', 'bus_count',
      ...,
      'unsignalized_int', 'urban', 'vehicle_failure', 'vehicle_towed',
      'vulnerable_road_user', 'vulnerable_road_user_fatal', 'wet_road',
      'work_zone', 'young_driver', 'geometry'],
      dtype='str', length=230)
```

EPSG:4326

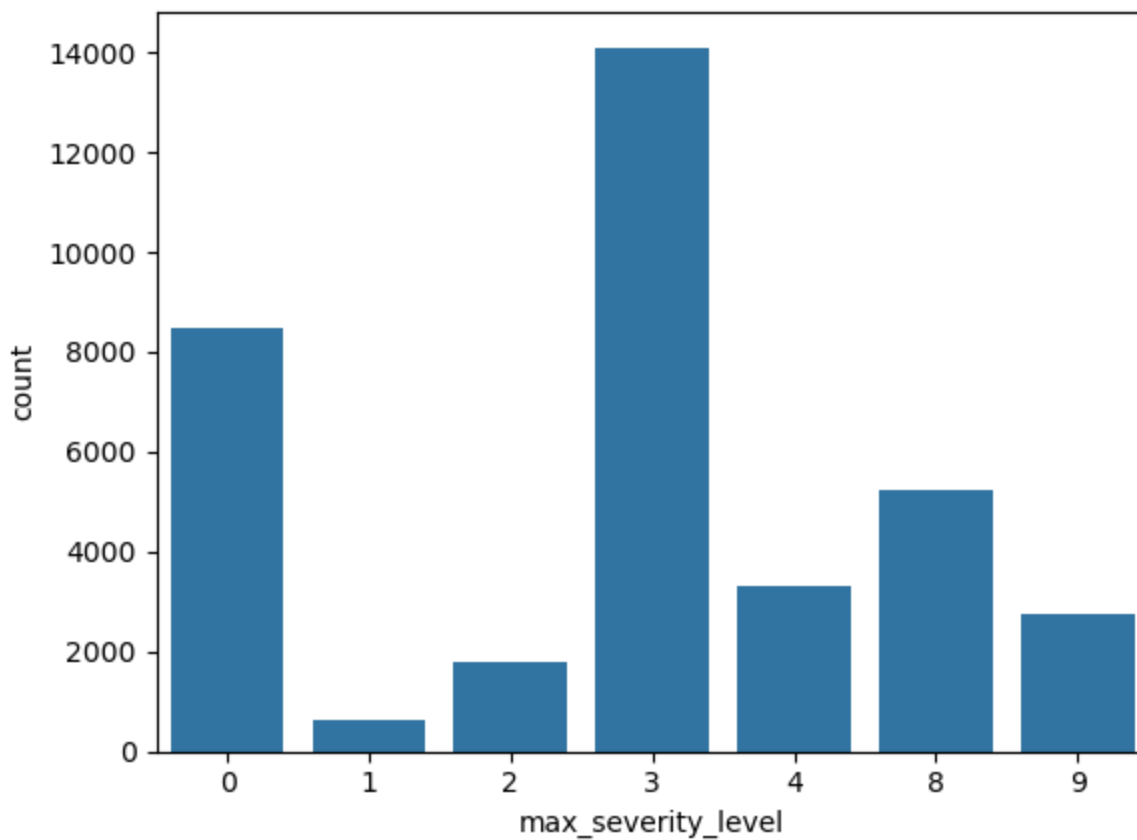
```
count    36303.000000
mean         3.481503
std          2.900757
min           0.000000
25%           1.000000
50%           3.000000
75%           4.000000
max           9.000000
```

Name: max_severity_level, dtype: float64

max_severity_level

```
3    14104
0     8494
8     5215
4     3328
9     2763
2     1779
1      620
```

Name: count, dtype: int64



Other fields of interest:

Weather1 and Weather2 have lots of NAs. I do not want to assume blanks mean fine conditions without stronger documentation.

Illumination is not very helpful, but `illumination_dark` is interesting: 0 = no and 1 = yes.

Time of day and hour of day are not reliable enough. We'll have to drop them. Police arrival time might have more signal, but still not enough for this version.

```
In [3]: # most do not have weather data. Are we to interpret that as fine conditions
print(crash_data.describe())
print(crash_data["weather1"].value_counts(dropna=False))
print(crash_data["weather2"].value_counts(dropna=False))

crashes_no_weather = crash_data[["weather1", "weather2"]].notna().any(axis=1)
print(f"crashes without weather1 or weather2 {crashes_no_weather}")

print(crash_data["illumination"].describe())
print(crash_data["illumination"].value_counts(dropna=False))
print(crash_data["illumination_dark"].value_counts(dropna=False))

# neither time_of_day nor hour_of_day are usable - too many NaNs
print(crash_data["hour_of_day"].value_counts(dropna=False))
print(crash_data["time_of_day"].value_counts(dropna=False))
print(crash_data["arrival_tm"].value_counts(dropna=False)) # maybe
```

	objectid	crn	automobile_count	belted_death_count	\
count	36303.000000	3.630300e+04	36303.000000	36303.000000	
mean	18152.000000	2.021927e+09	1.038564	0.000826	
std	10479.917748	1.403658e+06	0.870048	0.031480	
min	1.000000	2.020000e+09	0.000000	0.000000	
25%	9076.500000	2.021021e+09	0.000000	0.000000	
50%	18152.000000	2.022039e+09	1.000000	0.000000	
75%	27227.500000	2.023066e+09	2.000000	0.000000	
max	36303.000000	2.025050e+09	8.000000	2.000000	

	belted_susp_serious_inj_count	bicycle_count	bicycle_death_count	\
count	36303.000000	36303.000000	36303.000000	
mean	0.006749	0.031017	0.000771	
std	0.089586	0.173841	0.027762	
min	0.000000	0.000000	0.000000	
25%	0.000000	0.000000	0.000000	
50%	0.000000	0.000000	0.000000	
75%	0.000000	0.000000	0.000000	
max	3.000000	2.000000	1.000000	

	bicycle_susp_serious_inj_count	bus_count	chldpas_death_count	..
. \				
count	36303.000000	36303.000000	36303.000000	..
. mean	0.002424	0.021100	0.000083	..
. std	0.049176	0.145246	0.009090	..
. min	0.000000	0.000000	0.000000	..
. 25%	0.000000	0.000000	0.000000	..
. 50%	0.000000	0.000000	0.000000	..
. 75%	0.000000	0.000000	0.000000	..
. max	1.000000	2.000000	1.000000	..
.				

	unlicensed	unsignalized_int	urban	vehicle_failure	\
count	36303.000000	36303.000000	36303.000000	36303.000000	
mean	0.006308	0.198524	0.986172	0.073190	
std	0.079173	0.398894	0.116779	0.260451	
min	0.000000	0.000000	0.000000	0.000000	
25%	0.000000	0.000000	1.000000	0.000000	
50%	0.000000	0.000000	1.000000	0.000000	
75%	0.000000	0.000000	1.000000	0.000000	
max	1.000000	1.000000	1.000000	1.000000	

	vehicle_towed	vulnerable_road_user	vulnerable_road_user_fatal	\
count	36303.000000	36303.000000	36303.000000	
mean	0.682092	0.169793	0.007906	
std	0.465670	0.375456	0.088563	
min	0.000000	0.000000	0.000000	
25%	0.000000	0.000000	0.000000	
50%	1.000000	0.000000	0.000000	
75%	1.000000	0.000000	0.000000	
max	1.000000	1.000000	1.000000	

	wet_road	work_zone	young_driver
count	36303.000000	36303.000000	36303.000000
mean	0.135554	0.002947	0.098146
std	0.342319	0.054211	0.297516
min	0.000000	0.000000	0.000000
25%	0.000000	0.000000	0.000000
50%	0.000000	0.000000	0.000000
75%	0.000000	0.000000	0.000000
max	1.000000	1.000000	1.000000

[8 rows x 202 columns]

weather1

NaN 27719

03 6969

07 908

04 420

99 179

10 58

05 19

98 16

06 9

02 4

09 2

Name: count, dtype: int64

weather2

03 20093

NaN 11345

04 2077

07 1989

98 451

10 126

05 93

06 58

09 36

08 17

02 16

01 2

Name: count, dtype: int64

crashes without weather1 or weather2 27762

count 0

unique 0

top NaN

freq NaN

Name: illumination, dtype: object

illumination

None 36303

Name: count, dtype: int64

illumination_dark

0 21003

1 15300

Name: count, dtype: int64

hour_of_day

NaN 34341

15 125

17 122

16 114

18 114

20 101

00 99

```

21      95
22      93
19      90
13      84
08      81
09      80
23      79
12      79
01      79
11      78
14      70
02      69
10      67
07      61
06      52
03      50
05      44
04      35
99       1
Name: count, dtype: int64
time_of_day
NaN      34341
1600      24
2300      15
1530      14
2000      14
...
0651      1
0609      1
0827      1
2243      1
2140      1
Name: count, Length: 852, dtype: int64
arrival_tm
NaN      10019
1700      98
1730      86
1600      84
1630      82
...
0542      2
0314      2
0613      2
0412      2
0628      1
Name: count, Length: 1441, dtype: int64

```

```

In [4]: # while we can whittle our data down a bit, and drop what we do not need, we
print(crash_data.crs)
print(crash_data.centroid)

transformed_crash_path = "transformed-data/PennDOT Crash Data/crash_data_227
crash_data_2272 = to_2272_file(crash_data, transformed_crash_path)

print(crash_data_2272.crs)
print(crash_data_2272.centroid)

crash_data_2272.head()

```



```

EPSG:4326
0      POINT (-75.2477 39.96276)
1      POINT (-75.27763 39.97643)
2      POINT (-75.27917 39.97554)
3      POINT (-75.23992 39.94006)
4      POINT (-75.2615 39.87644)
...
36298  POINT (-75.02605 40.04363)
36299  POINT (-75.17295 39.92486)
36300  POINT (-75.11726 40.03164)
36301  POINT (-75.15036 39.9934)
36302  POINT (-75.11345 39.99645)
Length: 36303, dtype: geometry
read file
/var/folders/08/p4gj6k456sn0fw6n86vr5zp80000gn/T/ipykernel_52684/318750182
8.py:3: UserWarning: Geometry is in a geographic CRS. Results from 'centroid
' are likely incorrect. Use 'GeoSeries.to_crs()' to re-project geometries to
a projected CRS before this operation.

```

```

print(crash_data.centroid)
EPSG:2272
0      POINT (2669834.794 239229.185)
1      POINT (2661309.801 243969.631)
2      POINT (2660887.423 243634.198)
3      POINT (2672251.788 231023.074)
4      POINT (2666854.566 207686.117)
...
36298  POINT (2731030.16 270510.35)
36299  POINT (2691184.392 226031.331)
36300  POINT (2705635.051 265371.249)
36301  POINT (2696781.493 251174.056)
36302  POINT (2707084.731 252588.961)
Length: 36303, dtype: geometry

```

```

Out[4]:
  objectid      crn  arrival_tm  automobile_count  belted_death_count  belted_s
0         1  2020007295         NaN                4                  0
1         2  2020066105        1742                1                  0
2         3  2020067671         NaN                1                  0
3         4  2020086583        0458                1                  0
4         5  2021029069        2355                0                  0

```

5 rows x 230 columns

Centerline Street Data

Lane count: **we'll have to look elsewhere for lane count.**

Directionality: oneway values are B for both directions, FT for from-to digitization direction, and TF for the opposite.

Classification: class needs some cleaning. Here are the classifications:

- 0-Navy Yard
- 1-Expressways
- 2-Major Arterial
- 3-Minor Arterial
- 4-Collector
- 5-Local
- 6-Driveway
- 9-Low Speed Ramps
- 10-High Speed Ramps
- 12-Non Travable
- 14-City Boundary
- 13-???
- 15-Walking Connector
- 18-Traffic Controlled Crosswalks

We might not want to consider street types that are not driveable or have a near-zero possibility of crashes, like Driveways, Non-Traversable, City Boundary, Walking Connectors, and Traffic Controlled Crosswalks. Since I can't find a great source for every class, I'm going to plot and classify the data so I can see for myself, then make a call one way or another.

12 looks like things such as Forbidden Drive and walkways, though maintenance vehicles may access some of them.

Class 13 is a mixed bag. Legacy? At any rate, if we use this classification as a predictor, then we should probably exclude it.

Compare later to the Vision Zero High Injury Network centerline source: <https://opendataphilly.org/datasets/street-centerlines-for-vision-zero-high-injury-network-2017/>

```
In [5]: # centerline

center_line_data = gpd.read_file("raw-data/Street Centerline Data/Street_Cen
print(center_line_data.crs) # 4326

center_line_data.columns

print("oneway")
print(center_line_data["oneway"].value_counts())

print("pre_dir")
print(center_line_data["pre_dir"].value_counts())

print("seg_id")
```

```
print(center_line_data["seg_id"])

print("st_type")
print(center_line_data["st_type"])

print("class")

print(center_line_data.columns)

center_line_data["class"].value_counts()
```

```

EPSG:4326
oneway
oneway
B      22233
FT      9841
TF      9193
      4
Name: count, dtype: int64
pre_dir
pre_dir
      24860
N      5375
S      4364
W      3668
E      2856
NB      40
SB      38
EB      35
WB      35
Name: count, dtype: int64
seg_id
0      100002
1      100003
2      100004
3      100006
4      100007
...
41266   1180102
41267   1180103
41268   1180104
41269   1180105
41270   221259
Name: seg_id, Length: 41271, dtype: int32
st_type
0      AVE
1      AVE
2      AVE
3      PL
4      PL
...
41266   DR
41267   DR
41268   DR
41269   CT
41270   WAY
Name: st_type, Length: 41271, dtype: str
class
Index(['fnode_', 'tnode_', 'lpoly_', 'rpoly_', 'length', 'stcl2_', 'stcl2_i
d',
      'pre_dir', 'st_name', 'st_type', 'suf_dir', 'zip_left', 'zip_right',
      'l_f_add', 'l_t_add', 'r_f_add', 'r_t_add', 'st_code', 'l_hundred',
      'r_hundred', 'seg_id', 'oneway', 'class', 'responsibl', 'update_',
      'newsegdate', 'multi_rep', 'streetlabe', 'stname', 'objectid',
      'Shape__Length', 'geometry'],
      dtype='str')

```

```
Out[5]: class
4      15496
5      15300
3       4507
2       4487
12       421
1       420
14       167
10       164
9        116
6         94
13        79
15        11
18         9
Name: count, dtype: int64
```

```
In [6]: thirteens = center_line_data[center_line_data["class"] == 13]
thirteens.explore()

# these are all sorts of driveways, shopping centers, parks, universities. A
driveways = center_line_data[center_line_data["class"] == 6]
driveways.explore()

## before we manipulate, let's transform so we don't have to do it multiple
cl_2272_filepath = "transformed-data/Street Centerline Data/2272_centerlines"
center_line_2272 = to_2272_file(center_line_data, cl_2272_filepath)

exclude_classes = [6, 12, 13, 14, 15, 18]
# could show maps of others
center_line_subset = center_line_2272[~center_line_2272["class"].isin(exclude_classes)]
center_line_excluded = center_line_2272[center_line_2272["class"].isin(exclude_classes)]

center_line_master_2722 = center_line_subset.copy()
print(center_line_master_2722)
center_line_subset.explore(column="class", categorical=True, legend=True, crs="EPSG:31466")
```

read file

	fnode_	tnode_	lpoly_	rpoly_	length	stcl2_	stcl2_id	pre_dir	\
0	26705	26716	None	None	2862.844615	None	None		
1	26699	26704	None	None	935.023163	None	None		
2	26704	26705	None	None	122.512608	None	None		
3	26658	26672	None	None	735.818883	None	None		
4	26653	26668	None	None	735.209140	None	None		
...	...	...	...	...	...	...	...	...	...
41266	29256	29259	None	None	159.663507	None	None		
41267	29259	29258	None	None	218.496317	None	None		
41268	29259	29260	None	None	228.896914	None	None		
41269	29261	29260	None	None	390.822828	None	None		
41270	29713	29714	None	None	167.851340	None	None		

	st_name	st_type	...	class	responsibl	update_	\
0	BARTRAM	AVE	...	2	STATE	2004-05-18 00:00:00+00:00	
1	BARTRAM	AVE	...	2	STATE	1998-07-09 00:00:00+00:00	
2	BARTRAM	AVE	...	2	STATE	1998-07-09 00:00:00+00:00	
3	EASTWICK	PL	...	5	CITY	1998-07-09 00:00:00+00:00	
4	HARLEY	PL	...	5	CITY	2006-04-07 00:00:00+00:00	
...	...	...	...	...	...	...	...
41266	SUNFLOWER	DR	...	5	CITY	2011-08-30 00:00:00+00:00	
41267	MIMOSA	DR	...	5	CITY	2011-08-30 00:00:00+00:00	
41268	SUNFLOWER	DR	...	5	CITY	2011-08-30 00:00:00+00:00	
41269	BLUEBELL	CT	...	5	CITY	2011-08-30 00:00:00+00:00	
41270	EAGLES	WAY	...	5	PRIVATE	2011-04-18 00:00:00+00:00	

	newsegdate	multi_rep	streetlabe	stname	\
0	NaT	0.0	BARTRAM AVE	BARTRAM AVE	
1	NaT	0.0	BARTRAM AVE	BARTRAM AVE	
2	NaT	0.0	BARTRAM AVE	BARTRAM AVE	
3	NaT	0.0	EASTWICK PL	EASTWICK PL	
4	NaT	0.0	HARLEY PL	HARLEY PL	
...	...	...	...	...	...
41266	2008-02-28 00:00:00+00:00	0.0	SUNFLOWER DR	SUNFLOWER DR	
41267	2008-02-28 00:00:00+00:00	0.0	MIMOSA DR	MIMOSA DR	
41268	2008-02-28 00:00:00+00:00	0.0	SUNFLOWER DR	SUNFLOWER DR	
41269	2008-02-28 00:00:00+00:00	0.0	BLUEBELL CT	BLUEBELL CT	
41270	2011-04-18 00:00:00+00:00	0.0	EAGLES WAY	EAGLES WAY	

	objectid	Shape__Length	\
0	1	1137.344551	
1	2	371.888030	
2	3	48.720227	
3	4	292.693856	
4	5	292.440377	
...	...	...	...
41266	41267	63.661475	
41267	41268	87.173485	
41268	41269	91.299361	
41269	41270	155.870463	
41270	41271	66.860383	

	geometry
0	LINestring (2669013.871 209532.801, 2668959.63...
1	LINestring (2669588.606 210420.458, 2669083.44...
2	LINestring (2669083.444 209633.642, 2669013.87...
3	LINestring (2671108.574 213577.494, 2670709.97...
4	LINestring (2670902.343 213714.767, 2670496.88...

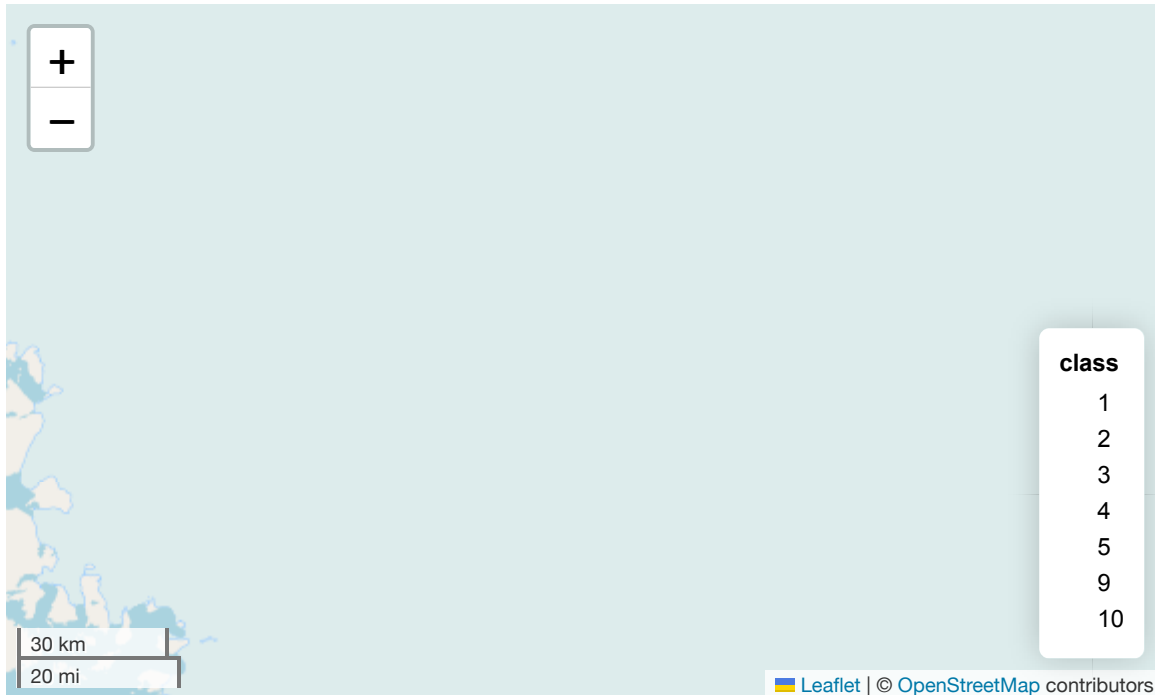
```

...
41266 LINESTRING (2741579.366 299180.315, 2741697.06...
41267 LINESTRING (2741697.063 299288.204, 2741836.97...
41268 LINESTRING (2741697.063 299288.204, 2741840.91...
41269 LINESTRING (2741576.097 299595.523, 2741597.34...
41270 LINESTRING (2689946.37 219271.105, 2689925.521...

```

[40490 rows x 32 columns]

Out [6]:



Indeed - we understand the centerline data better, but at the risk of not having speed limit or num of lanes. We'll definitely want speed limits.

Curbs, Cartways, and Road Width

Curbs/cartways are the source geometry for street-width estimation. The useful FCODE groups are:

- 1000 and 1001: cartway or paved travelway polygons.
- 1010-1012 and 1030-1032: concrete or grass islands, useful for understanding divided roads.

The final width calculation does not try to assign an entire cartway polygon to one street. Instead, it starts from each centerline segment and measures cross-sections through the cartway polygons.

Multipart cartways and divided roads are handled at the transect level. If a cross-section intersects multiple paved cartway pieces because an island, median, or multipart polygon splits the road, the code keeps those pieces separately as `n_parts`. It records `paved_width_ft` as the sum of the paved pieces, while `span_width_ft` measures the outermost-to-outermost span across all pieces. For the final model/app width, I use `cartway_paved_median_ft`, which avoids counting the physical island/median gap as paved road width.

In [7]: *# curbs with cartways*

```
curbs_with_cartways = gpd.read_file("raw-data/Curbs and Cartways/Curbs.geojson")
curbs_with_cartways_2272 = to_2272_file(curbs_with_cartways, "transformed-data/curbs_with_cartways_2272.geojson")

print(curbs_with_cartways_2272.head(10))
print(curbs_with_cartways_2272.crs) # 4326

print(curbs_with_cartways_2272.columns.tolist())
print(curbs_with_cartways_2272.head())
print(curbs_with_cartways_2272['fcode'].value_counts())

roadway_ids = [1000, 1001]
island_ids = [1010, 1011, 1012, 1030, 1031, 1032]

roadways = curbs_with_cartways_2272[curbs_with_cartways_2272["fcode"].isin(roadway_ids)]
islands = curbs_with_cartways_2272[curbs_with_cartways_2272["fcode"].isin(island_ids)]

islands.plot(column="fcode", categorical=True, legend=True, figsize=(10,10))
plt.show() # this looks correct - Roosevelt expressway; 76 + 95; airport + c
```


read file

	objectid	curbs_poly_	curbs_poly_id	fcode	elev_1	elev_2	elev_3	\
0	1	None	None	9999	1000.0	NaN	NaN	
1	2	None	None	1012	NaN	NaN	NaN	
2	3	None	None	1012	NaN	NaN	NaN	
3	4	None	None	1032	NaN	NaN	NaN	
4	5	None	None	1032	NaN	NaN	NaN	
5	6	None	None	1012	NaN	NaN	NaN	
6	7	None	None	1032	NaN	NaN	NaN	
7	8	None	None	1032	NaN	NaN	NaN	
8	9	None	None	1012	NaN	NaN	NaN	
9	10	None	None	1032	NaN	NaN	NaN	

		update_	add_date	chg_src	cp_id	Shape__Area	\
0	2013-01-25	00:00:00+00:00	None	None	26252	411.691406	
1	2013-01-25	00:00:00+00:00	None	None	26253	42.535156	
2	2013-01-25	00:00:00+00:00	None	None	26254	267.730469	
3	2013-01-28	00:00:00+00:00	None	None	26256	709.097656	
4	2013-01-28	00:00:00+00:00	None	None	26258	364.425781	
5	2013-01-25	00:00:00+00:00	None	None	26255	152.179688	
6	2013-01-28	00:00:00+00:00	None	None	26259	6508.601562	
7	2012-12-28	00:00:00+00:00	None	None	26240	1202.332031	
8	2013-01-28	00:00:00+00:00	None	None	26257	304.656250	
9	2012-12-27	00:00:00+00:00	None	None	26214	164.390625	

	Shape__Length	geometry
0	83.460408	POLYGON ((2662362.435 202967.987, 2662360.345 ...
1	52.152148	POLYGON ((2662403.197 203062.334, 2662402.502 ...
2	408.356301	POLYGON ((2662969.974 203178.704, 2662969.01 2...
3	234.473067	POLYGON ((2666736.587 206826.252, 2666739.972 ...
4	99.660293	POLYGON ((2665609.731 207010.407, 2665608.038 ...
5	58.536356	POLYGON ((2665474.422 207039.67, 2665474.943 2...
6	931.412654	POLYGON ((2665696.601 207066.18, 2665722.209 2...
7	349.656118	POLYGON ((2668523.665 207181.499, 2668549.619 ...
8	252.315599	POLYGON ((2666751.344 207184.013, 2666754.469 ...
9	51.315116	POLYGON ((2668401.124 207196.274, 2668402.121 ...

EPSG:2272

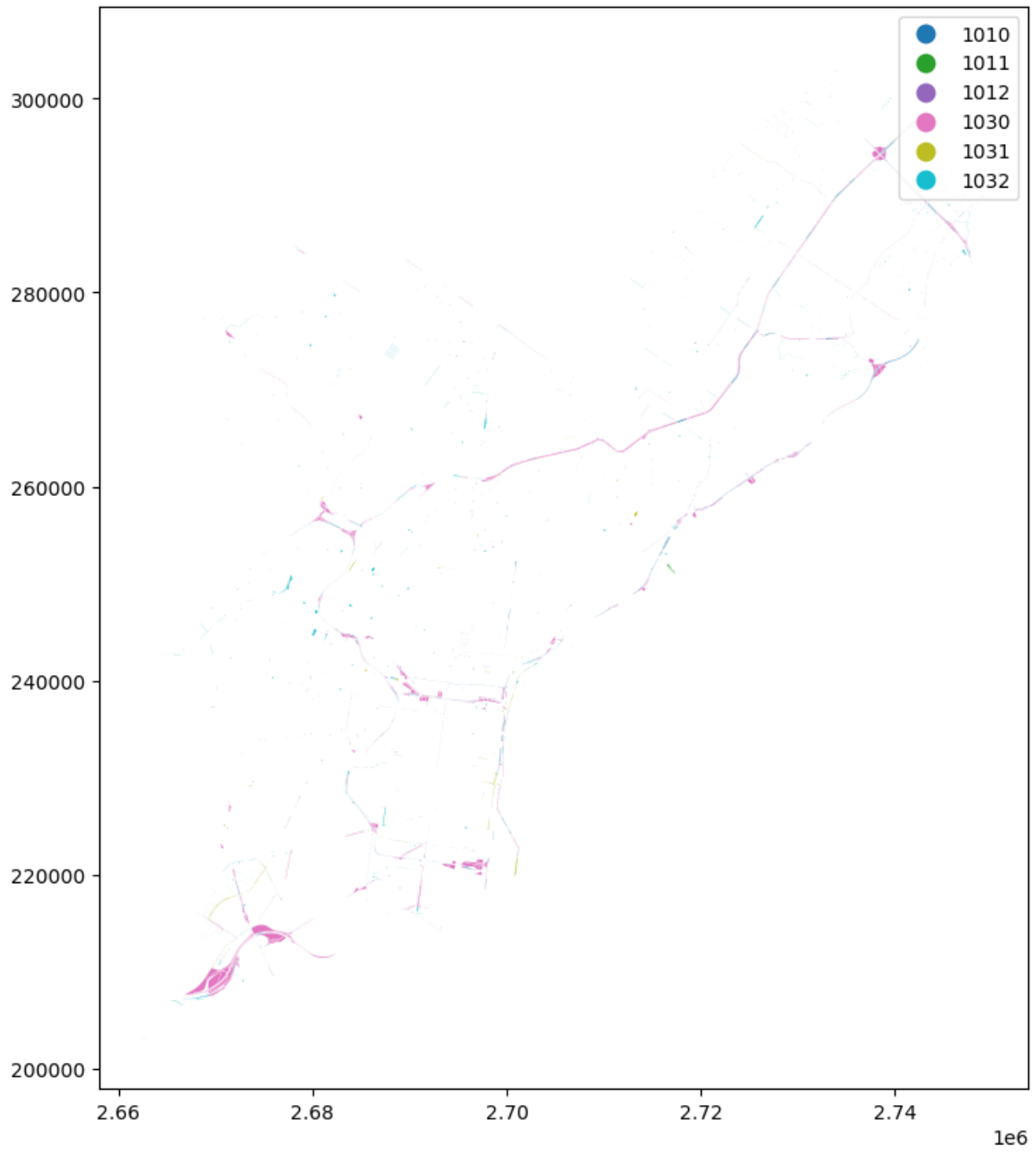
['objectid', 'curbs_poly_', 'curbs_poly_id', 'fcode', 'elev_1', 'elev_2', 'elev_3', 'update_', 'add_date', 'chg_src', 'cp_id', 'Shape__Area', 'Shape__Length', 'geometry']

	objectid	curbs_poly_	curbs_poly_id	fcode	elev_1	elev_2	elev_3	\
0	1	None	None	9999	1000.0	NaN	NaN	
1	2	None	None	1012	NaN	NaN	NaN	
2	3	None	None	1012	NaN	NaN	NaN	
3	4	None	None	1032	NaN	NaN	NaN	
4	5	None	None	1032	NaN	NaN	NaN	

		update_	add_date	chg_src	cp_id	Shape__Area	\
0	2013-01-25	00:00:00+00:00	None	None	26252	411.691406	
1	2013-01-25	00:00:00+00:00	None	None	26253	42.535156	
2	2013-01-25	00:00:00+00:00	None	None	26254	267.730469	
3	2013-01-28	00:00:00+00:00	None	None	26256	709.097656	
4	2013-01-28	00:00:00+00:00	None	None	26258	364.425781	

	Shape__Length	geometry
0	83.460408	POLYGON ((2662362.435 202967.987, 2662360.345 ...
1	52.152148	POLYGON ((2662403.197 203062.334, 2662402.502 ...
2	408.356301	POLYGON ((2662969.974 203178.704, 2662969.01 2...
3	234.473067	POLYGON ((2666736.587 206826.252, 2666739.972 ...

```
4      99.660293  POLYGON ((2665609.731 207010.407, 2665608.038 ...  
fcode  
9999    17355  
1000     2113  
1020     1302  
1010     1122  
1030      878  
1032      722  
1012      568  
1011      235  
1031      133  
8888       58  
0          45  
1001       26  
1021       22  
1022       13  
Name: count, dtype: int64
```



```

In [8]: ## State roads - let's see if we can incorporate

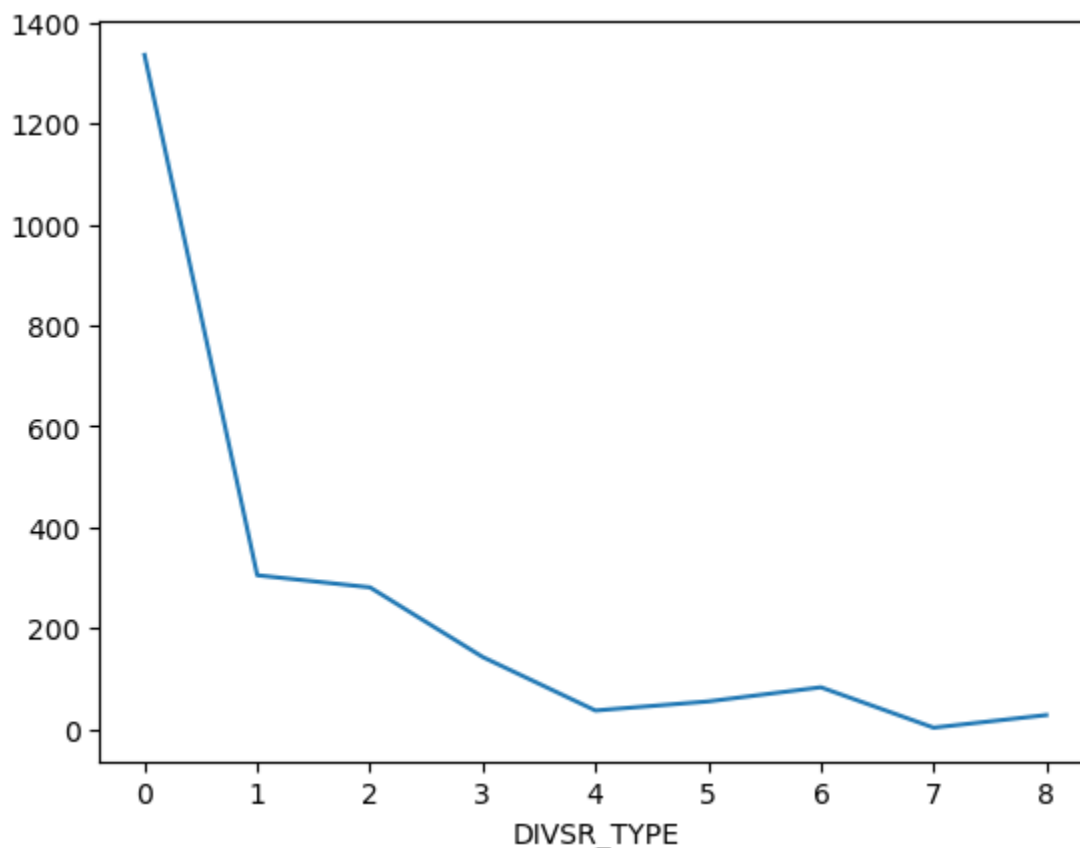
rmsseg = gpd.read_file("raw-data/PennDOT State Roads/RMSSEG_(State_Roads).ge

rmsseg = rmsseg[rmsseg["CTY_CODE"].astype(str) == "67"].to_crs(2272) # Phil
rmsseg.columns.to_list()
rmsseg["is_divided"] = (
    rmsseg["DIVSR_TYPE"].notna()
    & ~rmsseg["DIVSR_TYPE"].astype(str).str.strip().isin(["", "0", "00"])
) | (rmsseg["DIVSR_WIDTH"].fillna(0) > 0)

# bonus: keep the type for downstream modeling
rmsseg["median_type"] = rmsseg["DIVSR_TYPE"]
rmsseg.groupby("DIVSR_TYPE").size().plot()
rmsseg.plot(column="DIVSR_TYPE", categorical=True, legend=True, figsize=(25,
plt.show()
rmsseg["DIVSR_TYPE"].value_counts(dropna=False)
# first go is here, but we're seeing some false positives,. so we've gotta d
# https://gis.penndot.gov/BPR_PDF_FILES/Documents/Traffic/Highway_Statistics

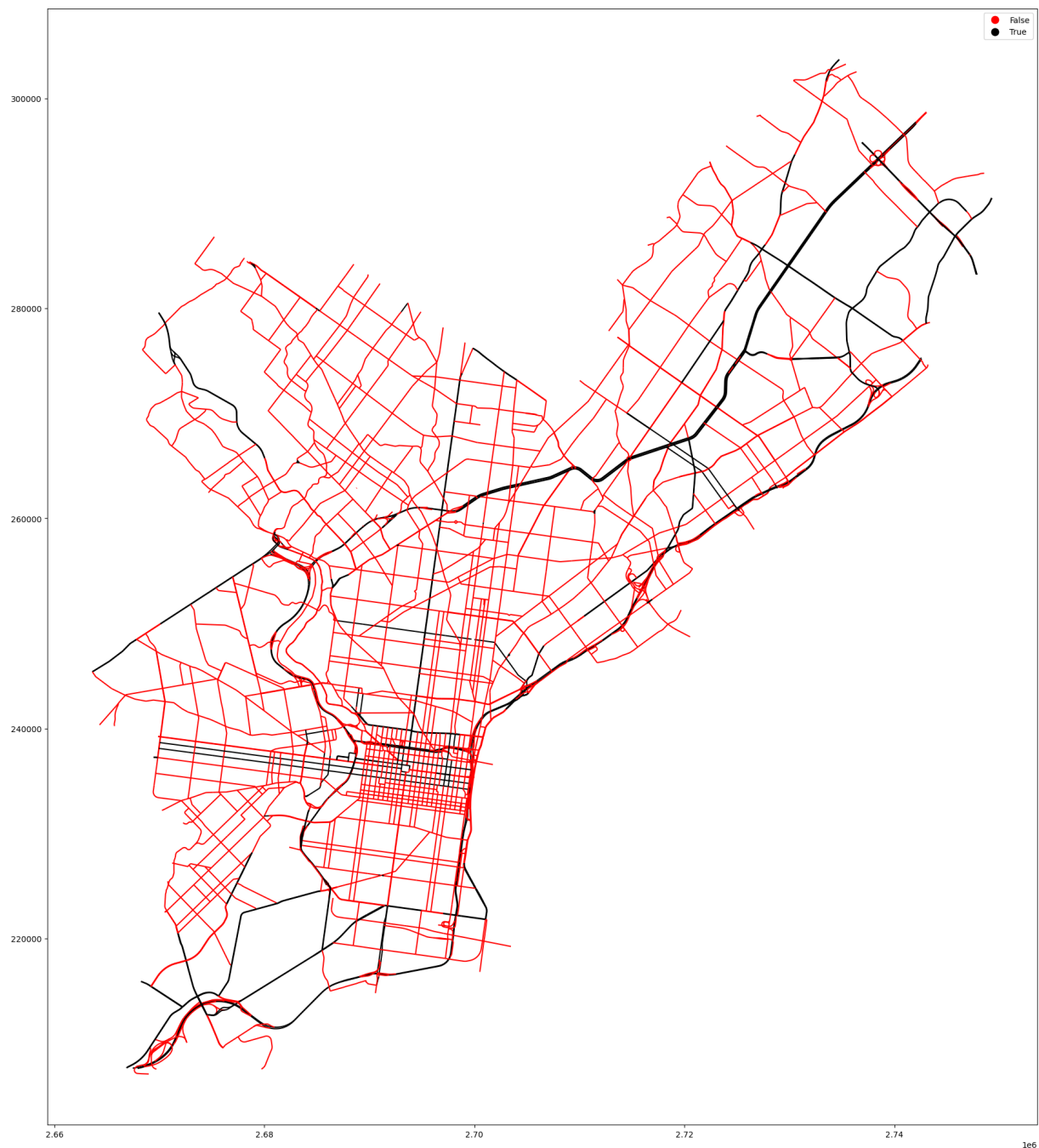
rmsseg["is_divided"] = rmsseg["DIVSR_TYPE"].astype(int) > 1 # still needs r
plt.show()
rmsseg.plot(column="is_divided", categorical=True, legend=True, cmap="flag",

```





Out[8]: <Axes: >



The PennDOT state-road attributes are useful for lanes, total width, and divided-road context, but they do not replace the cartway-width calculation. Many Philadelphia streets are not state roads, so the main width variable needs to be derived from city cartway and centerline geometry. The above state data can be used to complement or validate derived data.

Final Cartway Width Calculation: Centerline Transects

For each driveable centerline segment, I generated perpendicular transects at regular stations along the line, intersected those transects with cartway polygons, and summarized the measured widths by segment.

Process:

1. Filter curb/cartway polygons to paved travelway FCODEs 1000 and 1001.

2. Assign each centerline a temporary `cl_id`.
3. Walk along each centerline every 75 feet, trimming 50 feet from each end where possible to reduce intersection artifacts.
4. Estimate the local tangent from two nearby points on the centerline.
5. Rotate that tangent into a perpendicular unit vector: $\text{normal} = (-dy / d, dx / d)$.
6. Build a 250-foot cross-section centered on the station point.
7. Intersect the cross-section with cartway polygons.
8. Extract every line piece from the transect/cartway intersection. Multipart cartways or divided roads can produce multiple pieces on one transect.
9. Measure `paved_width_ft` as the sum of paved pieces, `span_width_ft` as the outermost-to-outermost span, and `n_parts` as the number of pieces.
10. Keep valid transects between 4 and 250 feet, then summarize per segment with median width and a confidence flag.

This route is implemented in `data_prep/widths.py` and enters the final analytical table through `compute_segment_widths(cl)`. The `model/app` field is `cartway_width_ft = cartway_paved_median_ft`.

```
In [9]: ROADWAY_FCODES = [1000, 1001]
        ISLAND_FCODES = [1010, 1011, 1012, 1030, 1031, 1032]

        roadways = curbs_with_cartways_2272[curbs_with_cartways_2272["fcode"].isin(ROADWAY_FCODES)]
        islands = curbs_with_cartways_2272[curbs_with_cartways_2272["fcode"].isin(ISLAND_FCODES)]

        def prep_gdf(gdf, epsg=2272):
            out = gdf.to_crs(epsg).copy()
            out = out[out.geometry.notna() & ~out.geometry.is_empty].copy()
            out["geometry"] = out.geometry.map(lambda g: make_valid(g) if not g.is_valid else g)
            return out[out.geometry.notna() & ~out.geometry.is_empty].copy()

        def line_parts(g):
            """Extract line pieces from an intersection geometry. Points add no width"""
            if g is None or g.is_empty:
                return []
            if g.geom_type == "LineString":
                return [g] if g.length > 0 else []
            if g.geom_type == "MultiLineString":
                return [p for p in g.geoms if p.length > 0]
            if g.geom_type == "GeometryCollection":
                parts = []
                for p in g.geoms:
                    parts.extend(line_parts(p))
                return parts
            return []
```

```
In [10]: def make_perpendicular_transects(
        centerlines, id_col="cl_id", spacing_ft=75, trim_ft=50,
        half_len_ft=125, tangent_delta_ft=5,
    ):
        rows = []
        for _, row in centerlines.iterrows():
            line = row.geometry
```

```

if line is None or line.is_empty or line.length <= 0:
    continue

L = float(line.length)
trim = min(float(trim_ft), max(0.0, (L - 1.0) / 2.0))

if L - 2 * trim < max(1.0, spacing_ft):
    stations = np.array([L / 2.0])
else:
    stations = np.arange(trim, L - trim + 1e-9, float(spacing_ft))
    if len(stations) == 0:
        stations = np.array([L / 2.0])

for j, s in enumerate(stations):
    p = line.interpolate(float(s))
    s0 = max(0.0, float(s) - float(tangent_delta_ft))
    s1 = min(L, float(s) + float(tangent_delta_ft))
    if s1 <= s0:
        continue

    a = line.interpolate(s0)
    b = line.interpolate(s1)
    dx, dy = b.x - a.x, b.y - a.y
    d = np.hypot(dx, dy)
    if d == 0:
        continue

    # Perpendicular unit vector. This is the key calculation:
    # tangent = (dx, dy), normal = (-dy, dx), then normalize.
    nx, ny = -dy / d, dx / d
    transect = LineString([
        (p.x - half_len_ft * nx, p.y - half_len_ft * ny),
        (p.x + half_len_ft * nx, p.y + half_len_ft * ny),
    ])
    rows.append({
        id_col: row[id_col],
        "transect_no": j,
        "station_ft": float(s),
        "geometry": transect,
    })

return gpd.GeoDataFrame(rows, geometry="geometry", crs=centerlines.crs)

```

```

In [11]: def measure_transect_widths(transects, surfaces, id_col="cl_id", grid_size_f
        """Measure cartway width along each transect.

        paved_width_ft = total length of transect pieces inside cartway polygons
        span_width_ft = outermost-to-outermost cartway span along the transect.
        """
        if transects.empty:
            return gpd.GeoDataFrame(
                columns=[id_col, "paved_width_ft", "span_width_ft"],
                geometry="geometry", crs=transects.crs,
            )

        surfaces = surfaces.to_crs(transects.crs)
        pairs = surfaces.sindex.query(transects.geometry, predicate="intersects")

        groups = {}

```

```

if pairs.size:
    pair_df = pd.DataFrame({"tx_pos": pairs[0], "surf_pos": pairs[1]})
    for tx_pos, g in pair_df.groupby("tx_pos", sort=False):
        groups[int(np.asarray(tx_pos).item())] = g["surf_pos"].to_numpy()

records = []
tx_geoms = transects.geometry.to_numpy()
surface_geoms = surfaces.geometry.reset_index(drop=True)

for tx_pos, t in enumerate(tx_geoms):
    surf_pos = groups.get(tx_pos)
    if surf_pos is None or len(surf_pos) == 0:
        paved_width = np.nan
        span_width = np.nan
        n_parts = 0
        center_covered = False
    else:
        surf = unary_union(list(surface_geoms.iloc[surf_pos].values))
        inter = shapely.intersection(t, surf, grid_size=grid_size_ft) if
        parts = line_parts(inter)
        paved_width = float(sum(p.length for p in parts)) if parts else
        n_parts = len(parts)

        if parts:
            projected_ends = []
            for p in parts:
                projected_ends.append(t.project(Point(p.coords[0])))
                projected_ends.append(t.project(Point(p.coords[-1])))
            span_width = float(max(projected_ends) - min(projected_ends))
        else:
            span_width = np.nan

        center_covered = bool(surf.covers(t.interpolate(t.length / 2.0)))

    records.append({
        id_col: transects.iloc[tx_pos][id_col],
        "transect_no": transects.iloc[tx_pos]["transect_no"],
        "station_ft": transects.iloc[tx_pos]["station_ft"],
        "paved_width_ft": paved_width,
        "span_width_ft": span_width,
        "n_parts": n_parts,
        "center_covered": center_covered,
        "geometry": t,
    })

return gpd.GeoDataFrame(records, geometry="geometry", crs=transects.crs)

```

```

In [12]: def summarize_widths(raw_transects, id_col="cl_id", prefix="cartway", min_wi
x = raw_transects.copy()
x["valid_width"] = (
    x["span_width_ft"].notna()
    & x["span_width_ft"].between(min_width_ft, max_width_ft)
)
good = x[x["valid_width"]].copy()

all_counts = x.groupby(id_col).size().rename(f"{prefix}_n_transects")
valid_counts = good.groupby(id_col).size().rename(f"{prefix}_n_valid")

stats = good.groupby(id_col).agg(

```



```

        span_median_ft=("span_width_ft", "median"),
        paved_median_ft=("paved_width_ft", "median"),
        parts_median=("n_parts", "median"),
        center_covered_share=("center_covered", "mean"),
    ).add_prefix(prefix + "_")

    summary = all_counts.to_frame().join(valid_counts).join(stats)
    summary[f"{prefix}_valid_share"] = (
        summary[f"{prefix}_n_valid"] / summary[f"{prefix}_n_transects"]
    )
    return summary.reset_index()

def widths_by_centerline(centerlines, surfaces, id_col="cl_id", spacing_ft=75):
    cl = centerlines.copy()
    if id_col not in cl.columns:
        cl = cl.reset_index(drop=True)
        cl[id_col] = cl.index

    cl = prep_gdf(cl)
    surfaces = prep_gdf(surfaces)

    cl_parts = cl.explode(index_parts=False).reset_index(drop=True)
    cl_parts = cl_parts[cl_parts.geometry.geom_type.isin(["LineString", "MultiLineString"])]

    transects = make_perpendicular_transects(
        cl_parts, id_col=id_col, spacing_ft=spacing_ft, trim_ft=trim_ft, half_len_ft=half_len_ft
    )
    raw_t = measure_transect_widths(transects, surfaces, id_col=id_col)
    summary = summarize_widths(raw_t, id_col=id_col)
    out = cl.merge(summary, on=id_col, how="left")
    return out, raw_t, transects

# Full run used by the final table builder:
# cl_for_widths = center_line_subset.reset_index(drop=True).copy()
# cl_for_widths["cl_id"] = cl_for_widths.index
# full_out, raw_width_transects, width_transects = widths_by_centerline(cl_for_widths,
# full_out["cartway_width_ft"] = full_out["cartway_paved_median_ft"]

```

```

In [13]: # Full run used by the final table builder. Speed:
# data_prep.widths.compute_segment_widths caches the output to a GPKG.
cl_for_widths = center_line_subset.reset_index(drop=True).copy()
cl_for_widths["cl_id"] = cl_for_widths.index

full_out, raw_width_transects, width_transects = widths_by_centerline(
    cl_for_widths,
    roadways,
    spacing_ft=75,
    trim_ft=50,
    half_len_ft=125,
)

full_out["cartway_width_ft"] = full_out["cartway_paved_median_ft"]
full_out["width_confidence"] = np.select(
    [
        full_out["cartway_n_valid"].fillna(0) >= 3,
        full_out["cartway_n_valid"].fillna(0) == 2,
        full_out["cartway_n_valid"].fillna(0) == 1,
    ],
    ["good", "okay", "low_one_or_two_transects"],
)

```

```

        default="missing",
    )

widths_for_final_table = full_out[[
    "seg_id",
    "cartway_width_ft",
    "cartway_paved_median_ft",
    "cartway_span_median_ft",
    "cartway_valid_share",
    "cartway_n_valid",
    "width_confidence",
]].copy()

widths_for_final_table.head()

```

```

Out[13]:
   seg_id  cartway_width_ft  cartway_paved_median_ft  cartway_span_median_ft  cart
0  100002          71.389208          71.389208          87.714916
1  100003          81.694486          81.694486          87.754561
2  100004         140.022466         140.022466         175.002870
3  100006          33.840768          33.840768          33.840768
4  100007          27.156240          27.156240          27.156240

```

Rejected Width Routes

I tried a few routes before the final transect approach:

- A direct roadway-polygon to centerline spatial join over-associated roads around intersections, islands, and divided roads.
- Polygon-level minimum-rotated-rectangle and area/length widths were useful sanity checks, but they measured cartway polygons rather than centerline segments.
- A buffered roadway-to-centerline match scored candidates by overlap and distance, but diagnostics were worse than the final centerline-first transect method.

The important distinction is that the final route starts from the centerline, cuts across the cartway, measures what that cross-section intersects, and then summarizes repeated measurements. That makes the width value segment-level by construction.

Traffic Calming Devices

This is exploratory. I inspect install dates and segment identifiers, then filter to devices installed before 2025 so future planned records do not leak into the analysis window.

```

In [14]: # calming devices

traffic_calming_devices = gpd.read_file("raw-data/Traffic Calming Devices/tr
traffic_calming_2272 = to_2272_file(traffic_calming_devices, "transformed-da

print("Calming: \n")
print(traffic_calming_devices.columns)

```

```
print(traffic_calming_devices.head(10))
print(traffic_calming_devices.crs) # 4326
# TODO: get unique values and add readable map
print(traffic_calming_devices.id.unique())
print(traffic_calming_devices.seg_id)
print(traffic_calming_devices.install_dt.sort_values()) # we have up to 2026

traffic_calming_devices_through_2024 = traffic_calming_devices[traffic_calmi
print(traffic_calming_devices_through_2024.install_dt.sort_values())
```

read file
Calming:

```
Index(['objectid', 'id', 'seg_id', 'install_dt', 'geometry'], dtype='str')
```

	objectid	id	seg_id	install_dt \
0	1	SC-331	423325.0	2023-08-01 04:00:00+00:00
1	2	SC-503	820354.0	2024-06-15 04:00:00+00:00
2	3	SC-199	860109.0	2015-05-11 00:00:00+00:00
3	4	SC-47	440754.0	2025-02-01 05:00:00+00:00
4	5	SC-295	960619.0	2022-05-21 04:00:00+00:00
5	6	SC-14	540865.0	2022-08-01 04:00:00+00:00
6	7	SC-296	1080224.0	2023-09-02 04:00:00+00:00
7	8	SC-881	401427.0	2025-11-01 04:00:00+00:00
8	9	SC-122	620972.0	2015-04-30 00:00:00+00:00
9	10	SC-200	1080035.0	2022-06-11 04:00:00+00:00

	geometry
0	POINT (-75.21172 39.9656)
1	POINT (-75.15205 40.05859)
2	POINT (-75.03605 40.05303)
3	POINT (-75.14631 39.96482)
4	POINT (-75.04045 40.09223)
5	POINT (-75.12096 39.99123)
6	POINT (-74.96714 40.10072)
7	POINT (-75.23542 39.97698)
8	POINT (-75.18378 40.01815)
9	POINT (-74.99383 40.10516)

EPSG:4326

<ArrowStringArray>

```
['SC-331', 'SC-503', 'SC-199', 'SC-47', 'SC-295', 'SC-14', 'SC-296',  
 'SC-881', 'SC-122', 'SC-200',  
 ...  
 'SC-672', 'SC-70', 'SC-215', 'SC-499', 'SC-324', 'SC-273', 'SC-322',  
 'SC-939', 'SC-940', 'sc-941']
```

Length: 745, dtype: str

0	423325.0
1	820354.0
2	860109.0
3	440754.0
4	960619.0

...	
1587	820093.0
1588	741285.0
1589	741291.0
1590	741255.0
1591	741291.0

Name: seg_id, Length: 1592, dtype: float64

1032	2009-04-01 00:00:00+00:00
128	2009-04-01 00:00:00+00:00
474	2009-04-01 00:00:00+00:00
1076	2011-09-01 00:00:00+00:00
840	2015-03-07 05:00:00+00:00

...	
146	2025-12-13 05:00:00+00:00
1280	2025-12-20 05:00:00+00:00
62	2025-12-20 05:00:00+00:00
785	2026-01-03 05:00:00+00:00
1165	2026-01-03 05:00:00+00:00

Name: install_dt, Length: 1592, dtype: datetime64[ms, UTC]

```

1032    2009-04-01 00:00:00+00:00
474    2009-04-01 00:00:00+00:00
128    2009-04-01 00:00:00+00:00
1076   2011-09-01 00:00:00+00:00
627    2015-03-07 05:00:00+00:00

```

...

```

399    2024-11-23 05:00:00+00:00
284    2024-11-23 05:00:00+00:00
1048   2024-11-30 05:00:00+00:00
751    2024-11-30 05:00:00+00:00
129    2024-11-30 05:00:00+00:00

```

Name: install_dt, Length: 1228, dtype: datetime64[ms, UTC]

Intersection Controls

Intersection controls are divided into three types: "Conventional", "All Way", and "Signalized". The first two are stop signs. "All Way" means all ways have a stop sign, at least we hope. Conventional means a non-all way stop, and signalized mean there's some sort of traffic signal. Unfortunately, for these last two controls we have no idea *which road* gets the stop sign or signal, though we can imagine it's the "more main" or heavier trafficked. For this analysis, we'll have to treat each type as a binary indicator - present or not. But present at what? There's no way to tell intersections from our centerline data alone. For this analysis, we'll buffer the intersection controls by 5m, and then see which centerlines intersect.

Attempted route: direct intersection-control join.

A direct spatial join from intersection controls to centerlines was considered, but it does not identify which approach road actually receives the stop sign or signal. The note remains to explain why this input stayed exploratory.

```

In [15]: intersection_controls = gpd.read_file("raw-data/Intersection Controls/Inters
intersection_controls_2272 = to_2272_file(intersection_controls, "transforme
print("intersections: \n")
print(intersection_controls_2272.columns)
print(intersection_controls_2272.head(10))
print(intersection_controls_2272.crs)
print(intersection_controls_2272.describe())
print(intersection_controls_2272.stoptype.unique())

# need to join now to... centerlines is the master
# I will siphon them off until im sure i can build it all up i guess
buf = intersection_controls_2272.copy().buffer(55)
buf.explore()
# Attempted route: direct intersection-control-to-centerline join.
# This was parked because the controls identify the intersection control typ
# but not which approach road receives the stop or signal.
# center_line_master_2722_ic_test = center_line_master_2722.sjoin(intersecti
# center_line_master_2722_ic_test.head(10)

```

```
read file
intersections:
```

```
Index(['objectid', 'stoptype', 'node_id', 'led_status', 'geometry'], dtype
='str')
```

	objectid	stoptype	node_id	led_status	geometry
0	1	Conventional	2	NaN	POINT (2733184.676 304859.04
1)					
1	2	Conventional	3	NaN	POINT (2733382.884 304711.63
9)					
2	3	Conventional	5	NaN	POINT (2733850.96 304361.29
8)					
3	4	Conventional	7	NaN	POINT (2734072.795 304195.26
2)					
4	5	All Way	8	NaN	POINT (2733030.26 304118.63
3)					
5	6	Conventional	10	NaN	POINT (2734312.538 304015.82
2)					
6	7	Conventional	12	NaN	POINT (2733108.868 303829.78
4)					
7	8	Conventional	13	NaN	POINT (2732443.974 303784.43
9)					
8	9	Conventional	14	NaN	POINT (2733774.506 303806.14
7)					
9	10	Signalized	15	Completed	POINT (2734750.014 303695.95
5)					

```
EPSG:2272
```

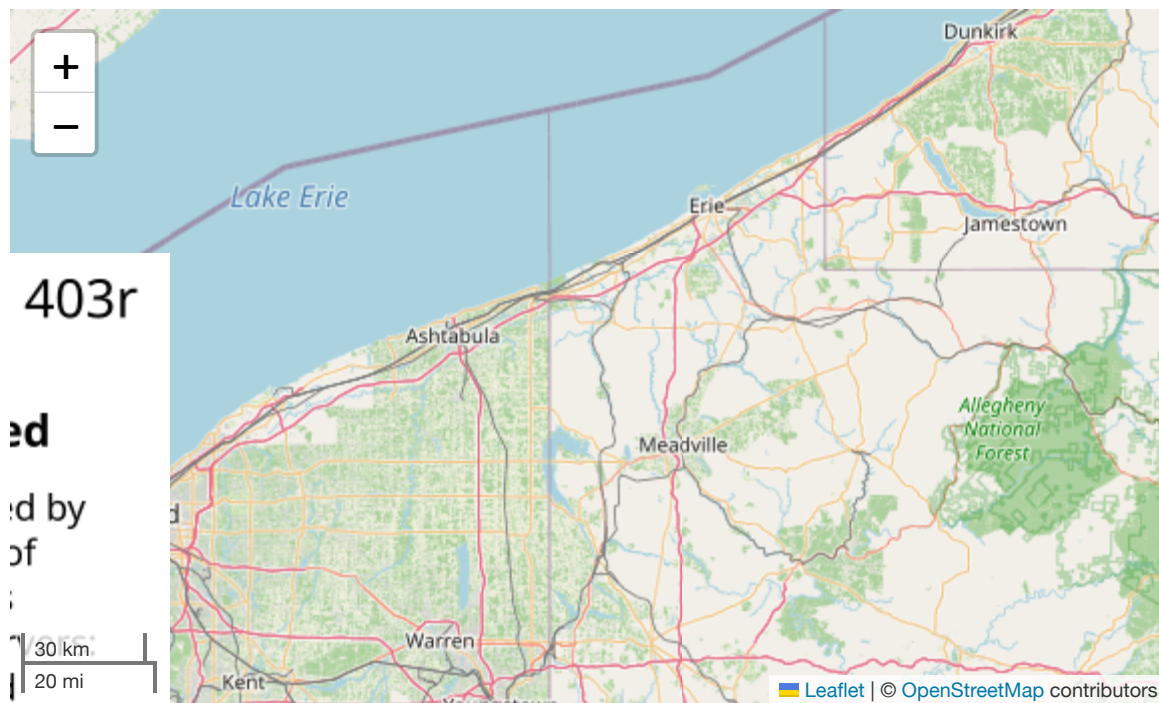
	objectid	node_id
count	16992.000000	16992.000000
mean	8496.500000	13622.318032
std	4905.312222	7858.818614
min	1.000000	2.000000
25%	4248.750000	6894.750000
50%	8496.500000	13572.500000
75%	12744.250000	20210.250000
max	16992.000000	99999.000000

```
<ArrowStringArray>
```

```
['Conventional', 'All Way', 'Signalized']
```

```
Length: 3, dtype: str
```

Out [15]:



In [16]:

```
# DVRPC Traffic counts

print("traffic counts: \n")

traffic_counts_path = "raw-data/DVRPC Traffic Count/dvrpc_2024_philly_traffic_counts.geojson"

if not os.path.exists("raw-data/DVRPC Traffic Count/dvrpc_2024_philly_traffic_counts.geojson"):
    url = "https://arcgis.dvrpc.org/portal/rest/services/transportation/traffic_counts/GeoJSON"
    params = {
        "where": "setyear=2024 AND co_name LIKE 'Phil%'",
        "outsr": 4326,
        "outfields": "*",
        "f": "geojson"
    }
    r = requests.get(url, params=params)
    counts = gpd.read_file(r.text)
    counts.to_file("raw-data/DVRPC Traffic Count/dvrpc_2024_philly_traffic_counts.geojson")
else:
    counts = gpd.read_file(traffic_counts_path)

print(counts.head(10))
print(counts.columns)
print(counts.crs) # 4326
```

traffic counts:

	objectid	recordnum	setdate	setyear	mcd	rdprefix	route
\							
0	121543	166189	17316288000000	2024	4210160104	NaN	NaN
1	123979	170740	17108064000000	2024	4210160105	I	95.0
2	123980	170741	17108064000000	2024	4210160105	I	95.0
3	123981	170743	17108064000000	2024	4210160105	I	95.0
4	123982	170744	17102880000000	2024	4210160105	I	95.0
5	123983	170745	17102880000000	2024	4210160105	I	95.0
6	123984	170747	17109792000000	2024	4210160105	NaN	NaN
7	123985	170748	17102880000000	2024	4210160105	I	95.0
8	123986	170749	17103744000000	2024	4210160105	I	95.0
9	123987	170750	17102880000000	2024	4210160105	I	95.0

	road	cntdir	fromlmt	\
0	levick st	north	jackson st	
1	off ramp	north	I 95 delaware expy nb	
2	on ramp	south	I 95 delaware expt nb	
3	on ramp	south	pa 291 bartram ave	
4	airport departures ramp	south	I 95 delaware expy nb	
5	off ramp	south	I 95 delaware expy sb	
6	on ramp	north	airport	
7	on ramp	south	island ave	
8	off ramp	north	I 95 delaware expy nb	
9	off ramp	south	I 95 delaware expy sb	

	tolmt	type	volume	outdir	indir	fc	sidewalk
\							
0	us 13 frankford ave	Manual Class	NaN	NaN	NaN	14	NaN
1	phila airport	Class	3543.0	NaN	NaN	11	NaN
2	arrivals rd	Class	10191.0	NaN	NaN	11	NaN
3	I 95 delaware expy sb	Class	3419.0	NaN	NaN	11	NaN
4	departures rc	Class	1293.0	NaN	NaN	11	NaN
5	pa 291 bartram ave	Class	3671.0	NaN	NaN	11	NaN
6	I 95 delaware expy nb	Class	3767.0	NaN	NaN	11	NaN
7	I 95 delaware expy sb	Class	13625.0	NaN	NaN	14	NaN
8	island ave/ platt bridge	Class	18907.0	NaN	NaN	14	NaN
9	pa 219 island ave	Class	11563.0	NaN	NaN	14	NaN

	co_name	mun_name	geometry
0	Philadelphia	North Delaware	POINT (-75.05477 40.02496)
1	Philadelphia	Lower South	POINT (-75.25105 39.87736)
2	Philadelphia	Lower South	POINT (-75.25297 39.87864)
3	Philadelphia	Lower South	POINT (-75.25315 39.88085)
4	Philadelphia	Lower South	POINT (-75.25073 39.88226)
5	Philadelphia	Lower South	POINT (-75.25058 39.88341)
6	Philadelphia	Lower South	POINT (-75.24202 39.88574)
7	Philadelphia	Lower South	POINT (-75.23725 39.8924)
8	Philadelphia	Lower South	POINT (-75.23238 39.89243)
9	Philadelphia	Lower South	POINT (-75.23028 39.89081)

```
Index(['objectid', 'recordnum', 'setdate', 'setyear', 'mcd', 'rdprefix',  
      'route', 'road', 'cntdir', 'fromlmt', 'tolmt', 'type', 'volume',  
      'outdir', 'indir', 'fc', 'sidewalk', 'co_name', 'mun_name', 'geometr  
y'],  
      dtype='str')
```

EPSG:4326

In [17]: # Tree canopy data - CRS = 2272


```

# according to the site - 1 is tree canopy
with rasterio.open("raw-data/Phila Land Cover Raster/PPR_LandCover_2018.gdb")
    print(src.crs) # 2722
    print(src.bounds)
    print(src.res) # 0.5 x 0.5
    print(src.shape)
    print(src.transform)
    print(src.tags())

# downsample in out_shape
data = src.read(1, out_shape=(src.height // 25, src.width // 25))

# mask to canopy only
canopy = np.where(data == 1, 1, 0) # why no y?

plt.figure(figsize=(10,10))
plt.imshow(canopy, cmap="Greens")
plt.title("2018 TC PH")
plt.axis("off")

plt.show()

```

EPSG:2272

BoundingBox(left=2645348.0, bottom=186454.0, right=2753588.0, top=307894.0)

(0.5, 0.5)

(242880, 216480)

| 0.50, 0.00, 2645348.00|

| 0.00,-0.50, 307894.00|

| 0.00, 0.00, 1.00|

{'RASTER_DATASET': 'landcover_2018_philadelphia'}



Elevation and Road Grade Calculation

Elevation comes from the Philadelphia 3-foot 2022 DEM. The final grade workflow is centerline-based, like the width workflow: sample elevation along each street segment, smooth the sampled profile, and summarize the grade per segment.

I tried two simpler routes first. Endpoint-only sampling was too sensitive to short segments and bad endpoint hits. Buffered zonal statistics around each segment produced noisy min/max ranges and too many unusable values. The final route samples points along the centerline itself and applies a rolling median to reduce bridge, overpass, and one-point raster artifacts.

```
In [18]: print("Elevation raster")
```

```

elevation_raster_path = "raw-data/Phila Elevation Raster/Philadelphia_dem_3f
NODATA_THRESHOLD = -1e30

with rasterio.open(elevation_raster_path) as src:
    print(src.crs)
    print(src.res)
    print(src.bounds)
    print(src.nodata) # float32 minimum sentinel, about -3.4e38

    # Downsample only for display, not for grade calculation.
    data = src.read(1, out_shape=(src.height // 10, src.width // 10))
    data_masked = np.ma.masked_less(data, NODATA_THRESHOLD)

plt.figure(figsize=(10, 10))
im = plt.imshow(data_masked, cmap="terrain")
plt.colorbar(im, label="Elevation (ft)")
plt.axis("off")
plt.show()

```

Elevation raster

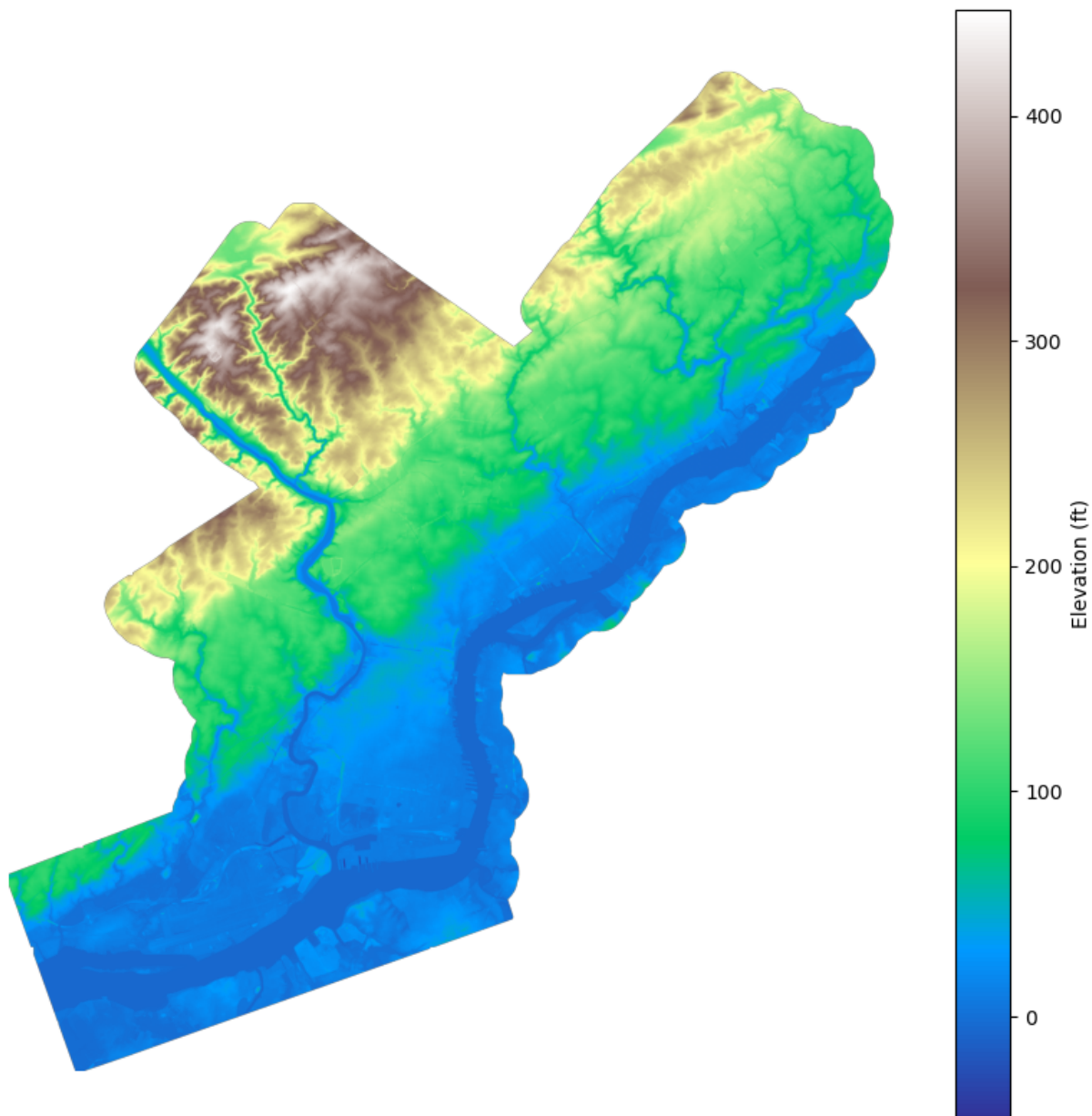
```

COMPD_CS["NAD83(2011) / Pennsylvania South (ftUS) + NAVD88 height (ftUS)", PROJCS["NAD83(2011) / Pennsylvania South (ftUS)", GEOGCS["NAD83(2011)", DATUM["NAD83_National_Spatial_Reference_System_2011", SPHEROID["GRS 1980", 6378137, 298.257222101, AUTHORITY["EPSG", "7019"]], AUTHORITY["EPSG", "1116"]], PRIMEM["Greenwich", 0, AUTHORITY["EPSG", "8901"]], UNIT["degree", 0.0174532925199433, AUTHORITY["EPSG", "9122"]], AUTHORITY["EPSG", "6318"]], PROJECTION["Lambert_Conformal_Conic_2SP"], PARAMETER["latitude_of_origin", 39.33333333333333], PARAMETER["central_meridian", -77.75], PARAMETER["standard_parallel_1", 40.96666666666667], PARAMETER["standard_parallel_2", 39.93333333333333], PARAMETER["false_easting", 1968500], PARAMETER["false_northing", 0], UNIT["US survey foot", 0.304800609601219, AUTHORITY["EPSG", "9003"]], AXIS["Easting", EAST], AXIS["Northing", NORTH], AUTHORITY["EPSG", "6565"]], VERT_CS["NAVD88 height (ftUS)", VERT_DATUM["North American Vertical Datum 1988", 2005, AUTHORITY["EPSG", "5103"]], UNIT["US survey foot", 0.304800609601219, AUTHORITY["EPSG", "9003"]], AXIS["Gravity-related height", UP], AUTHORITY["EPSG", "6360"]]]

```

(3.0, 3.0)

BoundingBox(left=2646000.0, bottom=187000.0, right=2754000.0, top=310000.0)
-3.4028234663852886e+38



Rejected Grade Routes

- **Endpoint sampling:** sample the first and last point of each segment, then compute $\text{abs}(\text{start_elev} - \text{end_elev}) / \text{length}$. This was intuitive and sometimes plausible, but produced extreme grades when either endpoint hit a bridge, wall, overpass, or bad raster edge.
- **Buffered zonal statistics:** buffer each segment and compute DEM min/max/range or percentile range inside the buffer. This captured too much surrounding terrain, especially near steep embankments and infrastructure, and was noisier than centerline sampling.

Those attempts were useful because they exposed the main failure mode: suspicious high-grade segments clustered near bridges, overpasses, ramps, Vine Street, and sharp elevation breaks.

```
In [19]: N_SAMPLES = 10  
         SMOOTH_WINDOW = 5
```

```

def sample_profile_along_line(line: LineString, src, n: int = N_SAMPLES):
    rows = []
    for i in range(n):
        frac = i / (n - 1)
        station_ft = frac * line.length
        p = line.interpolate(frac, normalized=True)
        val = next(src.sample([(p.x, p.y)]))[0]

        if val > NODATA_THRESHOLD:
            rows.append({
                "station_ft": station_ft,
                "elev_ft": float(val),
            })

    return rows

def get_grade_smoothed(line: LineString, src, n: int = N_SAMPLES, window: int = 5):
    """Return (grade_range_smooth, grade_median, grade_p90, grade_max)."""
    prof = sample_profile_along_line(line, src, n=n)
    if len(prof) < 2 or line.length == 0:
        return np.nan, np.nan, np.nan, np.nan

    g = pd.DataFrame(prof).sort_values("station_ft")
    g["elev_smooth_ft"] = (
        g["elev_ft"]
        .rolling(window=window, center=True, min_periods=2)
        .median()
    ).fillna(g["elev_ft"])

    dx = g["station_ft"].diff()
    dz = g["elev_smooth_ft"].diff()
    interval_grades = (dz / dx).abs().replace([np.inf, -np.inf], np.nan).dropna()

    # Main model/app field: total smoothed elevation range divided by segment length
    grade_range_smooth = (
        g["elev_smooth_ft"].max() - g["elev_smooth_ft"].min()
    ) / line.length

    if len(interval_grades) == 0:
        return grade_range_smooth, np.nan, np.nan, np.nan

    return (
        grade_range_smooth,
        interval_grades.median(),
        interval_grades.quantile(0.90),
        interval_grades.max(),
    )

```

```

In [20]: with rasterio.open(elevation_raster_path) as src:
    results = center_line_master_2722.geometry.apply(
        lambda line: get_grade_smoothed(line, src, n=N_SAMPLES, window=SMOOTH_WINDOW)
    )

```

```

center_line_master_2722["grade_range_smooth"] = results.apply(lambda x: x[0])
center_line_master_2722["grade_smooth_median"] = results.apply(lambda x: x[1])
center_line_master_2722["grade_smooth_p90"] = results.apply(lambda x: x[2])
center_line_master_2722["grade_smooth_max"] = results.apply(lambda x: x[3])

```

```
center_line_master_2722[[
    "seg_id",
    "grade_range_smooth",
    "grade_smooth_median",
    "grade_smooth_p90",
    "grade_smooth_max",
]] .head()
```

```
Out[20]:
```

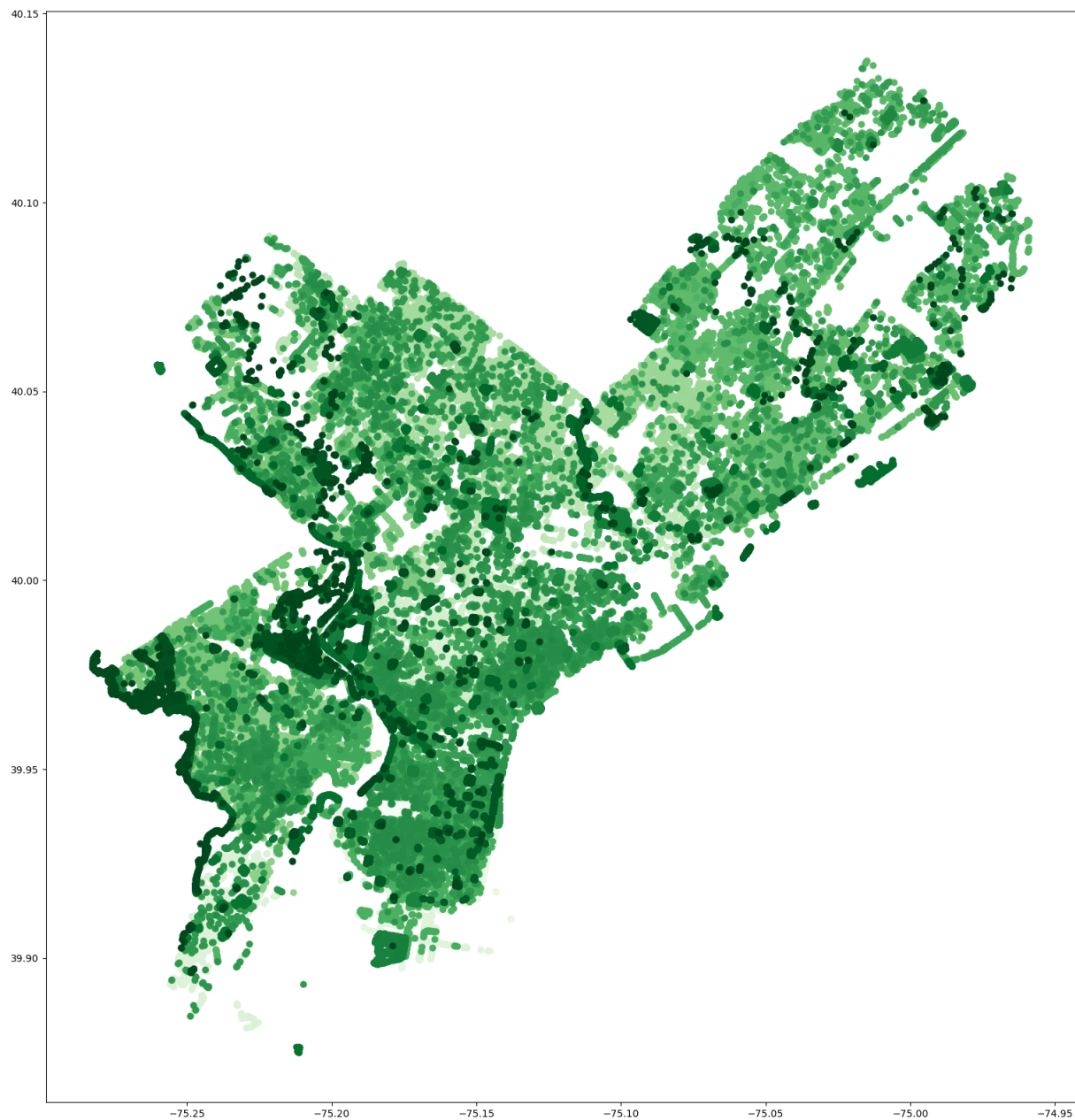
	seg_id	grade_range_smooth	grade_smooth_median	grade_smooth_p90	grade_si
0	100002	0.001356	0.001644	0.001994	
1	100003	0.002468	0.001901	0.007485	
2	100004	0.004261	0.003159	0.011152	
3	100006	0.003514	0.004789	0.008471	
4	100007	0.003637	0.004284	0.009830	

The final analytical table carries all four grade fields, but the main map/model field is `grade_range_smooth`. In plain terms, it is the segment's smoothed elevation range divided by segment length. Values around 0.03 are noticeable hills, values around 0.05 are steep, and very high values remain a data-quality warning to inspect bridges, ramps, overpasses, and very short fragments.

```
In [21]: print("PPD Street Trees ")
tree_data = gpd.read_file("raw-data/Phila Street Trees/ppr_tree_inventory_20
print(tree_data.crs) # 4326
tree_data.plot(cmap="Greens", ax=None, figsize=(20,20))
```

PPD Street Trees
EPSG:4326

```
Out[21]: <Axes: >
```



```
In [22]: print("Census info: ")

block_groups_path = "raw-data/Census/phila_block_groups_2024_4269.geojson"

if not os.path.exists(block_groups_path):
    block_groups = pygris.block_groups(state="PA", county="Philadelphia", year=2020)
    block_groups.to_file(block_groups_path, driver="GeoJSON")
else:
    block_groups = gpd.read_file(block_groups_path)

print(len(block_groups))
block_groups.head(10)
print(block_groups.crs) # 4269

census_api_key = os.environ.get("CENSUS_API_KEY")
if not census_api_key:
    raise RuntimeError("Set CENSUS_API_KEY before running Census cells")

cen = census.Census(census_api_key)
```



```

tract_data = cen.acs5.state_county_tract(
    fields=("NAME",
            "B19013_001E", # median income
            "B17001_002E", # poverty - hmmm None...
            "B01003_001E", # population
            "B08301_001E", # total commuters
            "B08301_003E", # drove alone
            "B08301_010E", # transit
            "B08301_019E", # walked
            "B08301_021E", # WFH
            "B02001_002E", # white alone
            "B25002_003E", # vacant units
            "B25058_001E", # median rent
            "B01002_001E"), # median age
    state_fips='42',
    county_fips="101",
    tract="*",
    year=2024
)

tract_data_df = pd.DataFrame(tract_data)
print("Tract: \n")
print(tract_data_df.head(10))

blockgroup_data = cen.acs5.state_county_blockgroup(
    fields=("NAME",
            "B19013_001E", # median income
            "B01003_001E", # population
            "B08301_001E", # total commuters
            "B08301_003E", # drove alone
            "B08301_010E", # transit
            "B08301_019E", # walked
            "B08301_021E", # WFH
            "B02001_002E", # white alone
            "B25002_003E", # vacant units
            "B25058_001E", # median rent
            "B01002_001E"), # median age
    state_fips='42',
    county_fips="101",
    blockgroup="*",
    year=2024
)

blockgroup_data_frame = pd.DataFrame(blockgroup_data)
print("Blockgroup: \n")

print(blockgroup_data_frame.head(10))

```


Census info:

1338

EPSG:4269

Tract:

	NAME	B19013_001E	\
0	Census Tract 1.01; Philadelphia County; Pennsy...	111148.0	
1	Census Tract 1.02; Philadelphia County; Pennsy...	101033.0	
2	Census Tract 2; Philadelphia County; Pennsylvania	102670.0	
3	Census Tract 3; Philadelphia County; Pennsylvania	102104.0	
4	Census Tract 4.01; Philadelphia County; Pennsy...	92182.0	
5	Census Tract 4.03; Philadelphia County; Pennsy...	167188.0	
6	Census Tract 4.04; Philadelphia County; Pennsy...	96024.0	
7	Census Tract 5; Philadelphia County; Pennsylvania	69677.0	
8	Census Tract 6; Philadelphia County; Pennsylvania	126033.0	
9	Census Tract 7.01; Philadelphia County; Pennsy...	85550.0	

	B17001_002E	B01003_001E	B08301_001E	B08301_003E	B08301_010E	\
0	96.0	1963.0	1646.0	308.0	331.0	
1	128.0	3172.0	2487.0	793.0	229.0	
2	714.0	3103.0	1653.0	304.0	195.0	
3	199.0	4727.0	3166.0	904.0	429.0	
4	481.0	2919.0	1816.0	238.0	311.0	
5	104.0	1010.0	743.0	76.0	72.0	
6	136.0	2588.0	1557.0	112.0	264.0	
7	361.0	3395.0	1650.0	279.0	566.0	
8	173.0	1923.0	1414.0	163.0	173.0	
9	221.0	2229.0	1639.0	189.0	308.0	

	B08301_019E	B08301_021E	B02001_002E	B25002_003E	B25058_001E	\
0	147.0	620.0	1577.0	97.0	1909.0	
1	284.0	1057.0	2559.0	268.0	2217.0	
2	365.0	729.0	1454.0	184.0	1629.0	
3	1126.0	523.0	2536.0	118.0	1914.0	
4	610.0	591.0	1410.0	302.0	1285.0	
5	140.0	364.0	830.0	98.0	2146.0	
6	459.0	685.0	2085.0	771.0	1920.0	
7	448.0	338.0	1580.0	134.0	1834.0	
8	673.0	381.0	1273.0	333.0	2264.0	
9	438.0	675.0	1772.0	246.0	2088.0	

	B01002_001E	state	county	tract
0	33.1	42	101	000101
1	35.5	42	101	000102
2	39.9	42	101	000200
3	34.1	42	101	000300
4	34.3	42	101	000401
5	39.3	42	101	000403
6	52.8	42	101	000404
7	33.7	42	101	000500
8	29.7	42	101	000600
9	31.9	42	101	000701

Blockgroup:

	NAME	B19013_001E	\
0	Block Group 1; Census Tract 1.01; Philadelphia...	-666666666.0	
1	Block Group 2; Census Tract 1.01; Philadelphia...	-666666666.0	
2	Block Group 3; Census Tract 1.01; Philadelphia...	-666666666.0	
3	Block Group 4; Census Tract 1.01; Philadelphia...	111220.0	

4	Block Group 5; Census Tract 1.01; Philadelphia...	99787.0
5	Block Group 1; Census Tract 1.02; Philadelphia...	97227.0
6	Block Group 2; Census Tract 1.02; Philadelphia...	-666666666.0
7	Block Group 3; Census Tract 1.02; Philadelphia...	103125.0
8	Block Group 4; Census Tract 1.02; Philadelphia...	183063.0
9	Block Group 5; Census Tract 1.02; Philadelphia...	93636.0

	B01003_001E	B08301_001E	B08301_003E	B08301_010E	B08301_019E	\
0	0.0	0.0	0.0	0.0	0.0	
1	0.0	0.0	0.0	0.0	0.0	
2	0.0	0.0	0.0	0.0	0.0	
3	1162.0	1023.0	193.0	164.0	56.0	
4	801.0	623.0	115.0	167.0	91.0	
5	1397.0	1121.0	489.0	42.0	155.0	
6	0.0	0.0	0.0	0.0	0.0	
7	879.0	659.0	133.0	116.0	44.0	
8	446.0	364.0	70.0	45.0	27.0	
9	450.0	343.0	101.0	26.0	58.0	

	B08301_021E	B02001_002E	B25002_003E	B25058_001E	B01002_001E	state	\
0	0.0	0.0	0.0	-666666666.0	-666666666.0	42	
1	0.0	0.0	0.0	-666666666.0	-666666666.0	42	
2	0.0	0.0	0.0	-666666666.0	-666666666.0	42	
3	452.0	889.0	73.0	1934.0	31.8	42	
4	168.0	688.0	24.0	1878.0	35.4	42	
5	404.0	1113.0	155.0	2077.0	33.7	42	
6	0.0	0.0	0.0	-666666666.0	-666666666.0	42	
7	346.0	712.0	0.0	2673.0	40.1	42	
8	202.0	397.0	0.0	2439.0	32.5	42	
9	105.0	337.0	113.0	1934.0	30.8	42	

	county	tract	block	group
0	101	000101		1
1	101	000101		2
2	101	000101		3
3	101	000101		4
4	101	000101		5
5	101	000102		1
6	101	000102		2
7	101	000102		3
8	101	000102		4
9	101	000102		5

```
In [23]: print("Land use")

land_usage = gpd.read_file("raw-data/Philly Land Use/Land_Use.geojson")
print(land_usage.crs) # 4326
```

Land use
EPSG:4326

Statistical Analysis and Models

The final modeling step uses the completed analytical table, one row per driveable street centerline segment. The response variable is `crash_count`, modeled as a count with a negative binomial GLM. I use `log_length` as an offset so longer segments are not treated the same as shorter ones.

The three-model structure matches the presentation:

1. **Model 1:** road design and exposure only.
2. **Model 2:** add environmental variables: tree canopy and grade.
3. **Model 3:** add interactions: canopy by cartway width, and grade by speed.

```
In [24]: model_data = gpd.read_file("transformed-data/Stash or final/philly_final_ana

def boolish_to_int(series):
    return series.map({True: 1, False: 0, "True": 1, "False": 0}).fillna(ser

for field in ["is_divided", "has_calming"]:
    if field in model_data.columns:
        model_data[field] = boolish_to_int(model_data[field])

cols_min = [
    "crash_count",
    "cartway_width_ft",
    "lanes_final",
    "maxspeed_final",
    "is_divided",
    "has_signal",
    "has_calming",
    "calming_device_count",
    "dvrpc_aadt",
    "canopy_pct",
    "grade_range_smooth",
    "median_income",
    "length",
    "class",
]

print(model_data[cols_min].isna().sum().sort_values(ascending=False))
```

```
lanes_final          39804
is_divided           39804
dvrpc_aadt           34267
cartway_width_ft      387
median_income         159
canopy_pct            20
crash_count           0
maxspeed_final        0
has_signal            0
has_calming           0
calming_device_count  0
grade_range_smooth    0
length               0
class                0
dtype: int64
```

```
In [25]: # Lane coverage is limited, so state-road lane data falls back to centerline
lanes_by_class = {
    1: 4, # Expressway
    2: 4, # Major Arterial
    3: 2, # Minor Arterial
    4: 2, # Collector
    5: 2, # Local
    9: 1, # Low Speed Ramp
```

```

    10: 2, # High Speed Ramp
}

model_data["lanes_final"] = model_data["lanes_final"].fillna(
    model_data["class"].map(lanes_by_class)
)

# Missing divided-road values mostly indicate non-state roads in this table.
model_data["is_divided"] = model_data["is_divided"].fillna(0)

# DVRPC AADT has sparse coverage. Keep a coverage flag and median-fill the v
model_data["has_aadt"] = model_data["dvrpc_aadt"].notna().astype(int)
model_data["dvrpc_aadt"] = model_data["dvrpc_aadt"].fillna(
    model_data["dvrpc_aadt"].median()
)

# Cartway width is mostly complete; fill remaining gaps by centerline class.
model_data["cartway_width_ft"] = model_data["cartway_width_ft"].fillna(
    model_data.groupby("class")["cartway_width_ft"].transform("median")
)

model_data["has_calming"] = (model_data["calming_device_count"] > 0).astype(int)

# The few segments without block-group context are boundary/industrial-edge
model_data = model_data.dropna(subset=["median_income"])

cols_final = [
    "crash_count",
    "cartway_width_ft",
    "lanes_final",
    "maxspeed_final",
    "is_divided",
    "has_signal",
    "has_calming",
    "dvrpc_aadt",
    "has_aadt",
    "canopy_pct",
    "grade_range_smooth",
    "length",
    "class",
]

model_df = model_data[cols_final].copy()
model_df["log_aadt"] = np.log1p(model_df["dvrpc_aadt"])
model_df["log_length"] = np.log(model_df["length"])
model_df["log_grade"] = np.log1p(model_df["grade_range_smooth"])

model_df.describe()

```

Out[25]:

	crash_count	cartway_width_ft	lanes_final	maxspeed_final	has_signal
count	40331.000000	40331.000000	40331.000000	40331.000000	40331.000000
mean	0.883539	38.548188	2.223922	28.952543	0.269123
std	2.321452	24.315464	0.644009	4.303382	0.443510
min	0.000000	4.249865	1.000000	25.000000	0.000000
25%	0.000000	25.534609	2.000000	25.000000	0.000000
50%	0.000000	33.215943	2.000000	30.000000	0.000000
75%	1.000000	42.392408	2.000000	30.000000	1.000000
max	63.000000	250.000000	7.000000	55.000000	1.000000

Why Negative Binomial?

Crash counts are sparse, non-negative count data, not continuous outcomes. A Poisson model would assume equal mean and variance, which is too restrictive for this kind of crash distribution. I used a negative binomial GLM because it allows overdispersion while keeping the model interpretable.

In [26]:

```
m1 = smf.glm(  
    formula=(  
        "crash_count ~ cartway_width_ft + lanes_final + maxspeed_final "  
        "+ is_divided + has_signal + has_calming "  
        "+ log_aadt + has_aadt"  
    ),  
    data=model_df,  
    family=sm.families.NegativeBinomial(),  
    offset=model_df["log_length"],  
).fit()  
  
m2 = smf.glm(  
    formula=(  
        "crash_count ~ cartway_width_ft + lanes_final + maxspeed_final "  
        "+ is_divided + has_signal + has_calming "  
        "+ log_aadt + has_aadt "  
        "+ canopy_pct + log_grade"  
    ),  
    data=model_df,  
    family=sm.families.NegativeBinomial(),  
    offset=model_df["log_length"],  
).fit()  
  
m3 = smf.glm(  
    formula=(  
        "crash_count ~ cartway_width_ft + lanes_final + maxspeed_final "  
        "+ is_divided + has_signal + has_calming "  
        "+ log_aadt + has_aadt "  
        "+ canopy_pct + log_grade "  
        "+ canopy_pct:cartway_width_ft "  
        "+ log_grade:maxspeed_final "  
    ),  
    data=model_df,
```

```
family=sm.families.NegativeBinomial(),
offset=model_df["log_length"],
).fit()

print("MODEL 1: Road design only")
print(m1.summary())
print("MODEL 2: Add environmental")
print(m2.summary())
print("MODEL 3: Add interactions")
print(m3.summary())
```

```
/Users/benjaminpolinsky/miniforge3/envs/roads/lib/python3.14/site-packages/s
tatsmodels/genmod/families/family.py:1367: ValueWarning: Negative binomial d
ispersion parameter alpha not set. Using default value alpha=1.0.
```

```
warnings.warn("Negative binomial dispersion parameter alpha not "
```

```
/Users/benjaminpolinsky/miniforge3/envs/roads/lib/python3.14/site-packages/s
tatsmodels/genmod/families/family.py:1367: ValueWarning: Negative binomial d
ispersion parameter alpha not set. Using default value alpha=1.0.
```

```
warnings.warn("Negative binomial dispersion parameter alpha not "
```

```
/Users/benjaminpolinsky/miniforge3/envs/roads/lib/python3.14/site-packages/s
tatsmodels/genmod/families/family.py:1367: ValueWarning: Negative binomial d
ispersion parameter alpha not set. Using default value alpha=1.0.
```

```
warnings.warn("Negative binomial dispersion parameter alpha not "
```

MODEL 1: Road design only

Generalized Linear Model Regression Results

```

=====
==
Dep. Variable:          crash_count    No. Observations:          403
31
Model:                  GLM           Df Residuals:              403
22
Model Family:          NegativeBinomial    Df Model:
8
Link Function:          Log             Scale:                  1.00
00
Method:                 IRLS            Log-Likelihood:         -4387
9.
Date:                   Sun, 03 May 2026    Deviance:                3875
9.
Time:                   12:58:21           Pearson chi2:             7.87e
+04
No. Iterations:         29                Pseudo R-squ. (CS):      0.31
93
Covariance Type:        nonrobust
=====
=====

```

```

=====
=====
                                coef      std err          z      P>|z|      [0.025
0.975]
-----

```

```

-----
Intercept                -9.1781      0.111      -82.395      0.000      -9.396
-8.960
is_divided[T.1.0]        0.0805      0.096       0.836      0.403      -0.108
0.269
cartway_width_ft          0.0072      0.000      19.245      0.000      0.006
0.008
lanes_final               0.1855      0.014      13.529      0.000      0.159
0.212
maxspeed_final            0.0513      0.002      23.072      0.000      0.047
0.056
has_signal                1.4384      0.018      79.795      0.000      1.403
1.474
has_calming               0.1350      0.050       2.682      0.007      0.036
0.234
log_aadt                  0.0303      0.013       2.397      0.017      0.006
0.055
has_aadt                  -0.0124     0.023     -0.531      0.595     -0.058
0.033
=====
=====

```

MODEL 2: Add environmental

Generalized Linear Model Regression Results

```

=====
==
Dep. Variable:          crash_count    No. Observations:          403
31
Model:                  GLM           Df Residuals:              403
20
Model Family:          NegativeBinomial    Df Model:
10
Link Function:          Log             Scale:                  1.00
00

```

```

Method:                IRLS    Log-Likelihood:        -4380
6.
Date:                  Sun, 03 May 2026    Deviance:        3861
2.
Time:                  12:58:21    Pearson chi2:        7.72e
+04
No. Iterations:        29    Pseudo R-squ. (CS):        0.32
18
Covariance Type:      nonrobust

```

```

=====
=====

```

	coef	std err	z	P> z	[0.025
0.975]					

Intercept	-9.0353	0.112	-80.470	0.000	-9.255
-8.815					
is_divided[T.1.0]	0.0935	0.096	0.970	0.332	-0.095
0.282					
cartway_width_ft	0.0068	0.000	18.116	0.000	0.006
0.008					
lanes_final	0.1890	0.014	13.770	0.000	0.162
0.216					
maxspeed_final	0.0505	0.002	22.683	0.000	0.046
0.055					
has_signal	1.4274	0.018	79.050	0.000	1.392
1.463					
has_calming	0.1420	0.050	2.818	0.005	0.043
0.241					
log_aadt	0.0270	0.013	2.142	0.032	0.002
0.052					
has_aadt	-0.0147	0.023	-0.628	0.530	-0.061
0.031					
canopy_pct	-0.7914	0.072	-10.917	0.000	-0.933
-0.649					
log_grade	-2.7630	0.712	-3.879	0.000	-4.159
-1.367					

```

=====
=====

```

MODEL 3: Add interactions

Generalized Linear Model Regression Results

```

=====
==
Dep. Variable:          crash_count    No. Observations:        403
31
Model:                  GLM    Df Residuals:        403
18
Model Family:          NegativeBinomial    Df Model:
12
Link Function:          Log    Scale:        1.00
00
Method:                IRLS    Log-Likelihood:        -4376
1.
Date:                  Sun, 03 May 2026    Deviance:        3852
2.
Time:                  12:58:21    Pearson chi2:        7.72e
+04
No. Iterations:        28    Pseudo R-squ. (CS):        0.32
33

```


Covariance Type: nonrobust

[0.025 0.975]		coef	std err	z	P> z
Intercept		-8.4234	0.121	-69.539	0.000
-8.661	-8.186				
is_divided[T.1.0]		0.0928	0.096	0.965	0.335
-0.096	0.281				
cartway_width_ft		0.0066	0.000	16.956	0.000
0.006	0.007				
lanes_final		0.1846	0.014	13.437	0.000
0.158	0.212				
maxspeed_final		0.0314	0.003	11.876	0.000
0.026	0.037				
has_signal		1.4265	0.018	78.995	0.000
1.391	1.462				
has_calming		0.1397	0.050	2.773	0.006
0.041	0.238				
log_aadt		0.0255	0.013	2.015	0.044
0.001	0.050				
has_aadt		-0.0091	0.023	-0.386	0.700
-0.055	0.037				
canopy_pct		-1.4095	0.150	-9.366	0.000
-1.704	-1.115				
log_grade		-51.2896	4.285	-11.969	0.000
-59.689	-42.891				
canopy_pct:cartway_width_ft		0.0166	0.003	4.939	0.000
0.010	0.023				
log_grade:maxspeed_final		1.5949	0.135	11.846	0.000
1.331	1.859				

```
In [27]: def lr_test(m_small, m_big, label):
    lr_stat = 2 * (m_big.llf - m_small.llf)
    df_diff = m_big.df_model - m_small.df_model
    p = 1 - chi2.cdf(lr_stat, df_diff)
    return {"Comparison": label, "LR": lr_stat, "df": df_diff, "p": p}

comparison = pd.DataFrame({
    "Model": ["M1: Road design", "M2: + Environment", "M3: + Interactions"],
    "AIC": [m1.aic, m2.aic, m3.aic],
    "Log-Likelihood": [m1.llf, m2.llf, m3.llf],
    "Pseudo-R2": [
        1 - m1.deviance / m1.null_deviance,
        1 - m2.deviance / m2.null_deviance,
        1 - m3.deviance / m3.null_deviance,
    ],
})

lr_results = pd.DataFrame([
    lr_test(m1, m2, "M1 vs M2: environment adds explanatory power"),
    lr_test(m2, m3, "M2 vs M3: interactions add explanatory power"),
])

display(comparison)
```

```
display(lr_results)
```

	Model	AIC	Log-Likelihood	Pseudo-R2
0	M1: Road design	87776.637726	-43879.318863	0.285867
1	M2: + Environment	87634.144894	-43806.072447	0.288566
2	M3: + Interactions	87548.050105	-43761.025052	0.290226

	Comparison	LR	df	p
0	M1 vs M2: environment adds explanatory power	146.492832	2	0.0
1	M2 vs M3: interactions add explanatory power	90.094789	2	0.0

Model Findings

The presentation findings came from these three negative binomial models. In the saved run, Model 1 had a Cox-Snell pseudo-R2 of about **0.3193**. Wider cartways, more inferred lanes, higher speed limits, traffic signals, traffic-calming presence, and AADT were positively associated with crash counts. The traffic-calming result is likely reactive: calming devices are often installed where crash risk is already known.

Model 2 improved modestly, to about **0.3218**, after adding canopy and grade. Both environmental variables were statistically significant and negatively associated with crashes: `canopy_pct` = **-0.7914** and `log_grade` = **-2.7630** in the saved run.

Model 3 improved again, to about **0.3233**, after adding interactions. The `canopy_pct:cartway_width_ft` interaction was positive, meaning canopy's apparent protective relationship is strongest on narrower streets and weakens as roads get wider. The `log_grade:maxspeed_final` interaction was also positive, meaning grade looks less protective, and potentially more hazardous, as speed context increases.

Likelihood-ratio tests in the saved run showed both added blocks improved fit: environment over road-design only, and interactions over the additive environment model. These remain correlational findings, not causal estimates.

Where the App Continues

This notebook stops at the analysis and data-preparation stage. The final app work happens in `app/` :

- `app/src/db.ts` initializes DuckDB-WASM and registers the parquet files.
- `app/src/mapQueries.ts` contains the browser-side SQL used by the map, peer comparisons, and story examples.
- `app/src/mapLayers.ts` and `app/src/map.ts` define the MapLibre sources, layers, legends, boundary, and interactions.

The notebook is still useful as the provenance trail: it shows how I inspected the raw

public datasets, made field decisions, and checked which inputs were worth carrying forward. The final app is hosted here: <https://benpolinsky.github.io/PHLCRSH/>.